



Panoptic: Next Core Findings & Analysis Report

2026-03-09

Table of contents

- [Overview](#)
 - [About C4](#)
- [Summary](#)
- [Scope](#)
- [Severity Criteria](#)
- [High Risk Findings \(3\)](#)
 - [\[H-01\] BuilderWallet .init\(\). is unprotected/re-initializable, enabling takeover and theft of builder fees](#)
 - [\[H-02\] Cross-contract reentrancy in liquidation enables conversion of phantom shares to real shares, draining CollateralTracker .assets](#)
 - [\[H-03\] Commission fees can always be bypassed](#)
- [Medium Risk Findings \(19\)](#)
 - [\[M-01\] Liquidations can be permanently blocked via .getLiquidationBonus\(\). _unsigned underflow \(Insolvent-but-unliquidatable accounts\)](#)
 - [\[M-02\] RiskEngine:: getRequiredCollateralAtTickSinglePosition\(\). fails to accumulate credits across multiple legs, leading to potential erroneous liquidations](#)
 - [\[M-03\] TWAP misweights EMAs in RiskEngine, anchoring liquidation price to slow EMA and letting insolvent accounts dodge liquidation](#)
 - [\[M-04\] Incorrect UPPER_118BITS_MASK _mask in OraclePackLibrary causes unexpected clearing of EMAs and lockMode in OraclePack](#)
 - [\[M-05\] Division-by-zero in long-leg collateral requirement can block solvency checks and dispatchFrom \(liquidation/force-exercise\) for tickSpacing==1 pools](#)
 - [\[M-06\] .dispatchFrom\(\). liveness DoS via StaleOracle : spot price manipulation blocks liquidations, force exercises, and premium settlements](#)
 - [\[M-07\] Liquidator can receive an inflated bonus against PLPs on PanopticPool. liquidate](#)
 - [\[M-08\] Wide-range short legs can revert solvency checks and block liquidations \(Errors.InvalidTick \)](#)
 - [\[M-09\] Incorrect collateral calculation for delayed swap strategies](#)
 - [\[M-10\] PLPs can withdraw assets needed by long positions, temporarily locking buyers](#)
 - [\[M-11\] An attacker can manipulate oracle easily](#)
 - [\[M-12\] Self-settlement via .dispatchFrom bypasses refund mechanism allowing underfunded debt settlement](#)
 - [\[M-13\] Intra-epoch rateAtTarget updates in CollateralTracker. updateInterestRate\(\). allow compounding interest rate errors](#)
 - [\[M-14\] State-price mismatch in liquidation](#)
 - [\[M-15\] Commission share-burn distribution is JIT-capturable when builderCode == 0 \(default\)](#)
 - [\[M-16\] Force exercise lacks caller-side bounds for exercise fee](#)

- [\[M-17\] High divergence check in `isSafeMode` is unreachable dead code providing false sense of security](#)
- [\[M-18\] Solvency tick divergence blind spot in `RiskEngine.getSolvencyTicks`](#)
- [\[M-19\] Withdrawing just before a bad debt event can increase losses for remaining liquidity providers](#)
- [Low Risk and Informational Issues](#)
 - [01 `validateCollateralWithdrawable\(\)` hardcodes `safeMode=0` ,bypassing stricter collateral requirements during volatile conditions](#)
 - [02 Missing zero shares check in `deposit\(\)` allows users to lose deposited assets](#)
 - [03 `CommissionPaid` event emits `protocolSplit` instead of `builderSplit` for `commissionPaidBuilder` parameter](#)
 - [04 `unlockPool\(\)` emits `GuardianSafeModeUpdated\(true\)` instead of `false` when lifting the lock](#)
 - [05 Memory copy of `premasByLeg` inside loop in `haircutPremia\(\)` causes redundant array copying on each iteration](#)
 - [06 `PanopticPool.onERC1155Received` accepts tokens from any ERC1155 contract without sender validation](#)
 - [07 `deposit\(\)` and `mint\(\)` are payable but do not refund ETH for ERC20 collateral, causing user funds to be trapped](#)
 - [08 Median tick calculation rounds toward zero instead of floor for negative values](#)
 - [09 `TokenId` validation allows `riskPartner` to reference inactive leg when index is 0](#)
 - [10 `Guardian.lockMode` not enforced in `getSolvencyTicks` allows weaker solvency checks during emergency lock](#)
 - [11 `TARGET\_UTILIZATION` `NatSpec` comment incorrectly states 90% instead of 66.67%](#)
 - [12 `NatSpec` comments in `Math.sol` incorrectly document boundary conditions for `getAmountsForLiquidity`](#)
- [Mitigation Review](#)
 - [Introduction](#)
 - [Mitigation Review Scope & Summary](#)
 - [Attacker can DoS liquidation by manipulating `currentTick` where the target position appears solvent](#)
- [Disclosures](#)

Overview

About C4

Code4rena (C4) is a competitive audit platform where security researchers, referred to as Wardens, review, audit, and analyze codebases for security vulnerabilities in exchange for bounties provided by sponsoring projects.

During the audit outlined in this document, C4 conducted an analysis of the Panoptic: Next Core smart contract system. The audit took place from December 19, 2025 to January 07, 2026.

Following the C4 audit, 4 wardens (Team [Valves](#) ([vesko210](#) and [Merulez99](#)), [edoscoba](#) and [Nyx](#)) reviewed the mitigations for all identified issues; the [mitigation review report](#) is appended below the audit report.

Final report assembled by Code4rena.

Summary

The C4 analysis yielded an aggregated total of 22 unique vulnerabilities. Of these vulnerabilities, 3 received a risk rating in the category of HIGH severity and 19 received a risk rating in the category of MEDIUM severity.

Additionally, C4 analysis included 108 QA reports compiling issues with a risk rating of LOW severity or

informational.

All of the issues presented here are linked back to their original finding, which may include relevant context from the judge and Panoptic team.

Considering the number of issues identified, it is statistically likely that there are more complex bugs still present that could not be identified given the time-boxed nature of this engagement. It is recommended that a follow-up audit and development of a more complex stateful test suite be undertaken prior to continuing to deploy significant monetary capital to production.

🔗 Scope

The code under review can be found within the [C4 Panoptic: Next Core repository](#), and is composed of 12 smart contracts written in the Solidity programming language and includes 6,356 lines of Solidity code.

The code in C4's Panoptic: Next Core repository was pulled from:

- Repository: <https://github.com/panoptic-labs/panoptic-next-core-private>
- Commit hash: `29980a740b67f3e5d9df9d96264a246b51fc7b6b`

🔗 Severity Criteria

C4 assesses the severity of disclosed vulnerabilities based on three primary risk categories: high, medium, and low/informational.

High-level considerations for vulnerabilities span the following key areas when conducting assessments:

- Malicious Input Handling
- Escalation of privileges
- Arithmetic
- Gas use

For more information regarding the severity criteria referenced throughout the submission review process, please refer to the documentation provided on [the C4 website](#), specifically our section on [Severity Categorization](#).

🔗 High Risk Findings (3)

🔗 [H-01] BuilderWallet `init()` is unprotected/re-initializable, enabling takeover and theft of builder fees

Submitted by [darf_tech](#), also found by [Ox04](#), [OxAura](#), [OxMirce](#), [Oxnbyc](#), [Oxzys](#), [agent001](#), [Agontuk](#), [Agrawain](#), [alaskanking](#), [AlexNiht](#), [allan31](#), [AnantaDeva](#), [arturtoros](#), [AuditorPraise](#), [axelot](#), [BlueSheep](#), [cheng9061](#), [ciphormalware](#), [critfinds](#), [Diavolo](#), [dtang](#), [edoscoba](#), [ElmInNyc99](#), [eternal1328](#), [forlz](#), [Funen](#), [Gakarot](#), [gegul](#), [gwumex](#), [happykilling](#), [ht111111](#), [humanitia](#), [IliveForTh1Sh1t](#), [Icarus](#), [Ituba](#), [jerry0422](#), [johnyfweh](#), [joicygiore](#), [K42](#), [Kaysoft](#), [kimnoic](#), [lamassu](#), [lazlosoot](#), [legat](#), [LimeiBBQ](#), [LoopGhost007](#), [lufP](#), [Manvita](#), [mehdi81](#), [Merkleboy](#), [merlin_san](#), [mibunna](#), [mohamedfahmy](#), [niffylord](#), [one](#), [piki](#), [qed](#), [Race](#), [Rifter](#), [rox_k](#), [s4id](#), [SarveshLimaye](#), [Spektor](#), [tkmk](#), [Trynax](#), [Tupaia](#), [Valves](#), [Vemus](#), [VulSight](#), [Web3Angel](#), [wilson9x1](#), [wuji](#), [Xylem](#), [zcai](#), and [zzebra83](#)

In `BuilderWallet`, the admin is stored in `builderAdmin` but `init()` has no access control and no “only-once” guard:

- `BuilderWallet` definition: `RiskEngine.sol` [#L2307](#)
- `init()` (unrestricted, overwrites `builderAdmin`): `RiskEngine.sol` [L2315](#)
- `sweep()` (only gated by `builderAdmin`): `RiskEngine.sol` [L2319](#)

Builder wallets are deployed by `BuilderFactory.deployBuilder(...)`, which calls `BuilderWallet(wallet).init(builderAdmin)` after CREATE2 deployment: `RiskEngine.sol` [L2371](#)

Because `init()` remains callable after deployment, any attacker can overwrite `builderAdmin` and then legitimately pass the `sweep()` authorization check.

🔗 Exploit Steps

1. Builder wallet is deployed via `deployBuilder(...)`: `RiskEngine.sol` [L2371](#)
2. Builder wallet accumulates ERC20 balances (fees/tokens).
3. Attacker calls `BuilderWallet.init(attacker)` to overwrite `builderAdmin`: `RiskEngine.sol` [L2315](#)
4. Attacker calls `BuilderWallet.sweep(token, attacker)` to drain balances: `RiskEngine.sol` [L2319](#)

🔗 Impact

Direct theft of all ERC20 balances held by any builder wallet (including protocol-distributed fees/shares).

🔗 Proof of Concept (minimal, runnable Foundry test)

[View detailed Proof of Concept](#)

[Panoptic mitigated:](#)

```
Protect builderWallet.init.
```

Status: Mitigation confirmed. Full details in reports from [Valves](#), [edoscoba](#), and [Nyx](#).

🔗 [H-02] Cross-contract reentrancy in liquidation enables conversion of phantom shares to real shares, draining CollateralTracker assets

Submitted by [ht111111](#), also found by [edoscoba](#), [fullstop](#), [johnnyf wesh](#), [legat](#), [qed](#), and [VulSight](#)

`CollateralTracker.sol` [L1359-L1360](#)

🔗 Impact

A malicious liquidator can exploit a cross-contract reentrancy vulnerability during the liquidation process to steal “phantom shares” (virtual liquidity used for solvency checks) and convert them into real, redeemable shares. This results in the complete drainage of assets from the `CollateralTracker` (the lending vault), causing a catastrophic loss of funds for liquidity providers.

🔗 Root Cause

The vulnerability stems from the interaction between three specific behaviors in the protocol:

1. **Inconsistent Revocation Timing:** `PanopticPool._liquidate` delegates phantom shares (amount `2^248 - 1`) to the liquidatee on both `token0` and `token1` collateral trackers simultaneously, but revokes them sequentially (`ct0` first, then `ct1`).
2. **Unsafe External Call:** `CollateralTracker.settleLiquidation` performs an external ETH refund call (`SafeTransferLib.safeTransferETH`) to the liquidator *before* the liquidation process is complete for the other token. This opens a reentrancy window.
3. **Accounting Flaw:** The `revoke` logic in `CollateralTracker` assumes any missing phantom shares were “consumed” (burned) by the protocol logic and compensates by increasing `_internalSupply`. It does not account for the possibility that phantom shares were transferred away.

🔗 Code Analysis

1. **The Trigger: External call** in `settleLiquidation`: In `CollateralTracker.sol`, when `settleLiquidation` is called with a non-negative bonus, it refunds any provided `msg.value` to the caller. This external call passes control to the attacker.

```
contracts/CollateralTracker.sol:

// In settleLiquidation:
} else {
    // ... (logic to revoke phantom shares for THIS tracker) ...

    // @audit-issue The external call here happens while OTHER trackers might still have active phantom shares
    if (msg.value > 0) SafeTransferLib.safeTransferETH(liquidator, msg.value);
}
```

2. **The Window: Sequential settlement** in `PanopticPool`: In `PanopticPool.sol`, settlements happen sequentially. When `ct0.settleLiquidation` is called, `ct1`'s phantom shares are active (delegated) but not yet revoked.

```
contracts/PanopticPool.sol:

// In _liquidate:
// ... Phantom shares delegated to liquidatee on both tokens ...

// 1. Settle token0 (Triggers ETH refund -> Reentrancy)
collateralToken0().settleLiquidation{value: msg.value}(...);

// 2. Settle token1 (Has not happened yet when reentrancy occurs)
collateralToken1().settleLiquidation(...);
```

3. **The Exploit: Transferring phantom shares**: During the reentrancy callback, the attacker calls `ct1.transferFrom`. The only check in `transferFrom` is `numberOfLegs == 0`. Since `_liquidate` burns positions before settlement, this check passes.

```
contracts/CollateralTracker.sol:

function transferFrom(address from, address to, uint256 amount) public override returns (bool) {
    // ...
    // @audit-issue This check passes during liquidation
    if (panopticPool().numberOfLegs(from) != 0) revert Errors.PositionCountNotZero();

    // @audit-issue Standard ERC20 transfer moves phantom shares without checks
    return ERC20Minimal.transferFrom(from, to, amount);
}
```

4. **The Validation: Incorrect supply compensation**: When control returns to `PanopticPool` and `ct1.settleLiquidation` is finally called, it calls `revoke`. Since the liquidatee's balance is now 0 (shares were transferred), the logic assumes they were consumed and validates them by increasing `_internalSupply`.

```
contracts/CollateralTracker.sol:

function revoke(address delegatee) external onlyPanopticPool {
    uint256 balance = balanceOf[delegatee];
    if (type(uint248).max > balance) {
        // @audit-issue Logic assumes missing shares were burned, permanently minting them
        _internalSupply += type(uint248).max - balance;
        balanceOf[delegatee] = 0;
    }
    // ...
}
```

Proof of Concept

A comprehensive Foundry test case demonstrates this vulnerability. The PoC performs the following steps:

1. Sets up a pool with liquidity in `token1`.
2. Creates a malicious liquidator contract.
3. Triggers liquidation with `msg.value = 1 wei` to force the ETH refund path.

4. In the `receive()` fallback, the liquidator transfers `ct1` phantom shares from the liquidatee to itself.
5. After liquidation completes, the liquidator holds valid shares and redeems them for the underlying assets.

[View detailed Proof of Concept](#)

🔗 Recommended mitigation steps

Option 1: Reentrancy guard (Recommended): Add a `nonReentrant` modifier to `CollateralTracker.settleLiquidation` to prevent re-entering the contract or other trackers during the settlement process.

Option 2: Restrict phantom share transfers: Modify `CollateralTracker.transferFrom` (and `transfer`) to explicitly forbid transferring shares if the balance indicates the presence of phantom shares.

```
function transferFrom(address from, address to, uint256 amount) public override returns (bool) {
+   // Prevent moving phantom shares
+   if (balanceOf[from] > type(uint248).max) revert Errors.PantomSharesAreNotTransferable();

    _accrueInterest(from, IS_NOT_DEPOSIT);
    if (panopticPool().numberOfLegs(from) != 0) revert Errors.PositionCountNotZero();
    return ERC20Minimal.transferFrom(from, to, amount);
}
```

Option 3: Move ETH refund: Move the ETH refund logic out of `CollateralTracker` and into `PanopticPool._liquidate` (specifically, at the very end of the function), ensuring all settlements and revocations are complete before making external calls.

[View detailed Proof of Concept](#)

[Panoptic mitigated:](#)

Enhance reentrancy protection and add protocol loss tracking.

Status: Mitigation confirmed. Full details in reports from [Valves](#), [edoscoba](#), and [Nyx](#).

🔗 [H-03] Commission fees can always be bypassed

Submitted by [prk0](#), also found by [ElmInNyc99](#), [peazzycole](#), and [zzebra83](#)

- `CollateralTracker.sol` [L1612-L1632](#)
- `PanopticPool.sol` [L1010-L1011](#)

🔗 Finding description

Original code block has been omitted. [View complete submission](#).

`CollateralTracker::settleBurn()` computes the `commissionFee` as the *minimum* between premium fee and notional fee.

`CollateralTracker::settleBurn()` is called in the following flows:

Flow 1 - Force Exercise:

```
PanopticPool::dispatchFrom()
→ PanopticPool::_forceExercise()
→ PanopticPool::_burnOptions()
→ CollateralTracker::settleBurn()
```

Flow 2 - Burn:

```
PanopticPool::dispatch()
```

```
→ PanopticPool::_burnOptions()
→ CollateralTracker::settleBurn()
```

Flow 3 - Liquidate:

```
PanopticPool::dispatchFrom()
→ PanopticPool::_liquidate()
→ PanopticPool::_burnAllOptionsFrom()
→ PanopticPool::_burnOptions()
→ CollateralTracker::settleBurn()
```

Flow 4 - Settle Premium:

```
PanopticPool::dispatchFrom()
→ PanopticPool::_settlePremium()
→ PanopticPool::_settleOptions()
→ CollateralTracker::settleBurn()
```

Flow 5 - Settle Premium:

```
PanopticPool::dispatch()
→ PanopticPool::_settleOptions()
→ CollateralTracker::settleBurn()
```

```
function _settleOptions(
    address owner,
    TokenId tokenId,
    uint128 positionSize,
    RiskParameters riskParameters,
    int24 currentTick
) internal {
    // call _updateSettlementPostBurn to settle the long premia or the short premia (only for self calling)
    LeftRightUnsigned[4] memory emptyCollectedByLegs;
    LeftRightSigned realizedPremia;
    unchecked {
        // cannot be miscast because currentTick is a int24
        (realizedPremia, ) = _updateSettlementPostBurn(
            owner, // owner
            tokenId, // tokenId
            emptyCollectedByLegs, // collectedByLeg
            positionSize, // positionSize
            riskParameters, // riskParameters
            LeftRightSigned.wrap(1).addToLeftSlot(1 + (int128(currentTick) << 2))
        );
    }
    // deduct the paid premium tokens from the owner's balance
    > collateralToken0().settleBurn(owner, 0, 0, 0, realizedPremia.rightSlot(), riskParameters); // @audit
    > collateralToken1().settleBurn(owner, 0, 0, 0, realizedPremia.leftSlot(), riskParameters); // @audit
}
```

In Flow 4 and Flow 5, `settleBurn()` is called from `_settleOptions()` - The supplied `longAmount`, `shortAmount`, and `ammDeltaAmount` parameters are all `0`. Therefore, we can expect the computed notional fee to also be `0`.

As mentioned above, the commission fee is taken from the *minimum* between notional fee and premium fee. As a result, the commission fee will always be computed to `0`, when `settleBurn()` is called from `_settleOptions()` (Flow 4 and Flow 5).

```
function settleBurn(
    address optionOwner,
    int128 longAmount,
    int128 shortAmount,
    int128 ammDeltaAmount,
    int128 realizedPremium,
    RiskParameters riskParameters
) external onlyPanopticPool returns (int128) {
    (, int128 tokenPaid, uint256 _totalAssets, uint256 _totalSupply) = _updateBalancesAndSettle(
        optionOwner,
        false, // isCreation = false
        longAmount,
        shortAmount,
        ammDeltaAmount,
        realizedPremium
    );
}
```

```

    );

    > if (realizedPremium != 0) {
        // SNIP
    }

    return tokenPaid;
}

```

`CollateralTracker::settleBurn()` also skips the commission fee computation altogether when `realizedPremium = 0`. Therefore, a user can always avoid commission fees by settling premium first, then burning.

🔗 Impact

Commission fees are always computed to 0 in flows that involve `_settleOptions()`. In addition, commission fees are skipped if `realizedPremium` is 0. This results in commission fees being skipped in many flows.

Users can avoid commission fees completely by settling premium, then burning, on exit.

🔗 Recommended mitigation steps

Consider removing the `realizedPremium != 0` check and consider deriving commission fee directly from premium fee, when `shortAmount` and `longAmount` are both `= 0`.

```

function settleBurn(
    address optionOwner,
    int128 longAmount,
    int128 shortAmount,
    int128 ammDeltaAmount,
    int128 realizedPremium,
    RiskParameters riskParameters
) external onlyPanopticPool returns (int128) {
    (, int128 tokenPaid, uint256 _totalAssets, uint256 _totalSupply) = _updateBalancesAndSettle(
        optionOwner,
        false, // isCreation = false
        longAmount,
        shortAmount,
        ammDeltaAmount,
        realizedPremium
    );

    - if (realizedPremium != 0) {
        uint128 commissionFee;
        // compute the minimum of the notionalFee and the premiumFee
        {
            uint128 commissionP;
            unchecked {
                commissionP = realizedPremium > 0
                    ? uint128(realizedPremium)
                    : uint128(-realizedPremium);
            }
            uint128 commissionFeeP = Math
                .mulDivRoundingUp(commissionP, riskParameters.premiumFee(), DECIMALS)
                .toUint128();
            uint128 commissionN = uint256(int256(shortAmount) + int256(longAmount)).toUint128();
            uint128 commissionFeeN;
            unchecked {
                commissionFeeN = Math
                    .mulDivRoundingUp(commissionN, 10 * riskParameters.notionalFee(), DECIMALS)
                    .toUint128();
            }

            + if (shortAmount == 0 && longAmount == 0) {
            +     commissionFee = commissionFeeP;
            +     } else {
            +         commissionFee = Math.min(commissionFeeP, commissionFeeN).toUint128();
            +     }
        }

        // SNIP
    - }

    return tokenPaid;
}

```

Proof of Concept

[View detailed Proof of Concept](#)

```
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 1.75s (100.30ms CPU time)

Ran 1 test suite in 2.04s (1.75s CPU time): 1 tests passed, 0 failed, 0 skipped (1 total tests)
```

The trace output above displays the last two interactions with `PanopticPool`, where Charlie settles his own premium, then burns the position.

In the first interaction (settle premium), the commission fee is computed to 0 for both `CollateralTracker0` and `CollateralTracker1`. This can be observed via the `CommissionPaid` event.

In the second interaction (burn), the commission fee is skipped. This can be confirmed due to the absence of the `CommissionPaid` event.

As a result, Charlie avoided paying any commission fees for his position.

[Panoptic mitigated:](#)

Prevent commission bypass.

Status: Mitigation confirmed. Full details in reports from [Valves](#), [edoscoba](#), and [Nyx](#).

Medium Risk Findings (19)

[M-O] Liquidations can be permanently blocked via `getLiquidationBonus()` unsigned underflow (Insolvent-but-unliquidatable accounts)

Submitted by [edoscoba](#), also found by [Oxnija](#), [Agontuk](#), [AlexNiht](#), [Bale](#), [caesar49](#), [dman](#), [eternal1328](#), [legat](#), [Micky042](#), [OxNoble](#), [ProngsDev](#), [VulSight](#), and [Zitifethefirst](#)

- `RiskEngine.sol` [L516](#)
- `RiskEngine.sol` [L1029](#)
- `PanopticMath.sol` [L669-L689](#)
- `PanopticPool.sol` [L1540](#)

Finding description

`PanopticPool.dispatchFrom()` routes to liquidation when `RiskEngine.isAccountSolvent()` returns false at all liquidation ticks. The solvency check is intentionally *not* based on total cross-collateral value: it applies a utilization-dependent `crossBufferRatio` that can reduce cross-margining to ~0 at high utilization, forcing a strict per-token solvency requirement.

However, `RiskEngine.getLiquidationBonus()` computes a single cross-denominated “collateral balance” and “required threshold” using `PanopticMath.getCrossBalances(...)`, then applies the documented formula:

```
liquidationBonus = min(collateralBalance / 2, required - collateralBalance)
```

This implementation assumes `required >= collateralBalance` and performs the subtraction in unsigned arithmetic:

```
uint256 bonusCross = Math.min(balanceCross / 2, thresholdCross - balanceCross);
```

When liquidation is triggered due to *distribution insolvency* (e.g., `token1` balance is below requirement while `token0` has a large surplus, and cross-margining is disabled by `crossBufferRatio`), it is possible and realistic for `balanceCross > thresholdCross` even though `isAccountSolvent()` is false. In that case, `thresholdCross -`

`balanceCross` underflows and reverts, which bubbles up and reverts the entire liquidation.

🔗 Impact

- **Liquidation DoS:** accounts that the protocol deems liquidatable can become **unliquidatable**, breaking the core safety mechanism.
- **Systemic risk escalation:** undercollateralized portfolios can remain open and cannot be force-closed, increasing the likelihood of PLP losses and blocking recovery mechanisms that depend on liquidation completing successfully.

🔗 Recommended mitigation steps

- Make `getLiquidationBonus()` robust to `balanceCross >= thresholdCross` by clamping the deficit:
 - ```
uint256 deficitCross = thresholdCross > balanceCross ? thresholdCross - balanceCross : 0;
```
  - ```
uint256 bonusCross = Math.min(balanceCross / 2, deficitCross);
```
- Align liquidation bonus computation with the protocol's solvency semantics:
 - Either compute "effective cross collateral" using the same `crossBufferRatio` methodology used in `isAccountSolvent()`, or
 - Derive bonus from per-token deficits/surpluses directly (rather than a single cross-total), so distribution-insolvent accounts cannot trigger a revert in the liquidation path.
- Keep (and extend) regression tests covering liquidation paths where cross-margining is disabled by high utilization.

🔗 Proof of Concept

A runnable PoC:

- `test/foundry/core/LiquidationBonusUnderflowPoC.t.sol`

Run it with:

- `forge test --match-path test/foundry/core/LiquidationBonusUnderflowPoC.t.sol -vvv`

[View detailed Proof of Concept](#)

[Panoptic mitigated:](#)

No more underflow for liquidation bonus calculation with no cross-margin.

Status: Mitigation confirmed. Full details in reports from [Valves](#), [edoscoba](#), and [Nyx](#).

🔗 [M-02] `_RiskEngine::_getRequiredCollateralAtTickSinglePosition()` fails to accumulate credits across multiple legs, leading to potential erroneous liquidations

Submitted by [joicygiore](#), also found by [Oxhunter20](#), [OxMirce](#), [Oxnija](#), [agent001](#), [Agontuk](#), [Coachmike](#), [Diavolo](#), [edoscoba](#), [eightzerofour](#), [eternal1328](#), [ewah](#), [forlz](#), [Funen](#), [Hajime](#), [Hakeem is here](#), [happykilling](#), [hecker_trieu_tien](#), [Incognito](#), [kimnoic](#), [lamassu](#), [legat](#), [NexusAudits](#), [sahuang](#), and [VulSight](#)

`RiskEngine.sol` [L1324-L1334](#)

🔗 Finding description

The function `RiskEngine::_getRequiredCollateralAtTickSinglePosition()` is responsible for calculating both the required collateral (`tokenRequired`) and the user's applicable credit amount (`credits`) for a single position at a given tick. These values are critical inputs to the protocol's solvency and liquidation logic.

When a `tokenId` contains multiple legs, and more than one leg satisfies the following condition:

- The function computes a credit amount for each qualifying leg. However, instead of accumulating these values, the implementation overwrites the `credits` variable on each iteration:

```
function _getRequiredCollateralAtTickSinglePosition(
    TokenId tokenId,
    uint128 positionSize,
    int24 atTick,
    int16 poolUtilization,
    bool underlyingIsToken0
) internal view returns (uint256 tokenRequired, uint256 credits) {
    uint256 numLegs = tokenId.countLegs();

    unchecked {
        for (uint256 index = 0; index < numLegs; ++index) {
            // bypass the collateral calculation if tokenType doesn't match the requested token (underlyingIsToken0)
            if (tokenId.tokenType(index) != (underlyingIsToken0 ? 0 : 1)) continue;

            if (tokenId.width(index) == 0 && tokenId.isLong(index) == 1) {
                LeftRightUnsigned amountsMoved = PanopticMath.getAmountsMoved(
                    tokenId,
                    positionSize,
                    index,
                    false
                );

                credits = tokenId.tokenType(index) == 0
                    ? amountsMoved.rightSlot()
                    : amountsMoved.leftSlot();
            }
            // Increment the tokenRequired accumulator
            tokenRequired += _getRequiredCollateralSingleLeg(
                tokenId,
                index,
                positionSize,
                atTick,
                poolUtilization
            );
        }
    }
}
```

As a result, only the credit contribution from the last matching leg is preserved, while all prior credits are silently discarded.

🔗 Impact

Because `credits` is undercounted, the `RiskEngine` underestimates the user's effective margin when evaluating solvency.

This incorrect value propagates through the following critical call path:

```
isAccountSolvent()
→ _getMargin()
→ _getTotalRequiredCollateral()
→ _getRequiredCollateralAtTickSinglePosition()
```

Consequently, an account that is actually solvent may be incorrectly classified as insolvent, leading to unexpected and unjustified liquidations.

🔗 Recommended mitigation steps

1. Treat credits as an accumulator and sum the credit contributions from all qualifying legs, rather than

overwriting the value.

2. Clearly distinguish between:

- Per-leg credit contributions
- The total credit amount for a multi-leg position

3. Add unit tests covering multi-leg scenarios, including:

- Multiple `width == 0 && isLong == true` legs
- Mixed `tokenType` configurations

4. (Optional) Update inline documentation or comments to clearly define the expected semantics of `credits`, reducing the risk of future regressions.

Proof of Concept

Please add the following test content to a new file under `test/foundry/core/` (e.g., `test/foundry/core/poc_getRequiredCollateralAtTickSinglePosition.t.sol`) and execute it:

[View detailed Proof of Concept](#)

Output:

```
Ran 2 tests for test/foundry/core/poc_getRequiredCollateralAtTickSinglePosition.t.sol:Poc
[PASS] test__getRequiredCollateralAtTickSinglePosition_one() (gas: 7827689)
[PASS] test__getRequiredCollateralAtTickSinglePosition_two() (gas: 7493324)
```

[Panoptic mitigated:](#)

Combine credit amounts for multileg `tokenId` s.

Status: Mitigation confirmed. Full details in reports from [Valves](#), [edoscoba](#), and [Nyx](#).

[M-03] TWAP misweights EMAs in RiskEngine, anchoring liquidation price to slow EMA and letting insolvent accounts dodge liquidation

Submitted by [mrdafidi](#), also found by [Oxanony](#), [Ahmedsec](#), [Charming](#), [Diavolo](#), [dtang](#), [edoscoba](#), [ewah](#), [Funen](#), [fx](#), [hecker](#), [trieu](#), [tien](#), [hiia](#), [IliveForTh1Sh1t](#), [Incogknito](#), [KineticsOfWeb3](#), [M_o7amed_T](#), [Manosh19](#), [mohamedfahmy](#), [Neo3141](#), [niffylord](#), [odeili](#), [one](#), [qwgkol](#), [SAGEisbuilding](#), and [Wakei](#)

This issue was also found with [V12](#).

- `RiskEngine.sol` [L836](#)
- `OraclePack.sol` [L209-L224](#)

Root cause

`twapEMA` destructures `getEMAs()` as `(int256 eonsEMA, int256 slowEMA, int256 fastEMA, )` even though `getEMAs()` returns `(spotEMA, fastEMA, slowEMA, eonsEMA, medianTick)` (`RiskEngine.sol` [line 836](#) vs `OraclePack.sol` [lines 209-224](#)). The weighting `int24((6 * fastEMA + 3 * slowEMA + eonsEMA) / 10)` therefore becomes 60% slow, 30% fast, 10% spot and ignores the eons EMA entirely.

Impact

`PanopticPool.getTWAP()` uses this TWAP for liquidation/force-exercise checks (`PanopticPool.sol` [lines 1944-1948](#)). `dispatchFrom` requires insolvency at all ticks `[spot, twap, latest, current]` before proceeding (`PanopticPool.sol` [lines 1368-1438](#)). Because the TWAP is anchored to the slow EMA, rapid adverse moves leave the TWAP near stale prices, so unhealthy accounts can appear solvent at that tick and

block liquidation/force-exercise (`NotMarginCalled`), delaying cleanup and risking larger bad debt or mispriced exercises/settlements.

🔗 Recommended mitigation steps

Deconstruct EMAs in the correct order and keep the intended 60/30/10 fast/slow/eons weighting.

```
// contracts/RiskEngine.sol
function twapEMA(OraclePack oraclePack) external pure returns (int24) {
    ( , int256 fastEMA, int256 slowEMA, int256 eonsEMA, ) = oraclePack.getEMAs();
    return int24((6 * fastEMA + 3 * slowEMA + eonsEMA) / 10);
}
```

Add a regression test ensuring `twapEMA` weights fast/slow/eons and does not depend on spot.

🔗 Proof of Concept

```
forge test --match-path test/foundry/core/RiskEngine/RiskEngineTwapEMAPoC.t.sol --match-test twapEMA_weights_fast_slow_eons_not_spot
```

[View detailed Proof of Concept](#)

Panoptic mitigated:

Use correct returned ema order in `twapEMA`.

Status: Mitigation confirmed. Full details in reports from [Valves](#), [edoscoba](#), and [Nyx](#).

🔗 [M-04] Incorrect UPPER_118BITS_MASK mask in OraclePackLibrary, causes unexpected clearing of EMAs and lockMode in OraclePack

Submitted by [joicygiore](#), also found by [Oxnija](#), [Oxspryon](#), [41rR4z0r](#), [Agontuk](#), [anonymousjoe](#), [aster](#), [axelot](#), [Race](#), [random1106](#), [slvDev](#), [SoarinSkySagar](#), [theboiledcorn](#), [Tupaia](#), and [Vinay](#)

OraclePack.sol [L64-L65](#)

🔗 Finding description and impact

When the `OraclePackLibrary::rebaseOraclePack()` function is called, the `UPPER_118BITS_MASK` mask is intended to clear the lower 118 bits of the `referenceTick` and `residuals` in `OraclePack`. However, the design of the mask is incorrect. The mask should only clear specific portions of the `referenceTick` and `residuals`, but due to the incorrect mask length, it inadvertently clears the `EMAs` and `lockMode` data as well.

This error results in the accidental overwriting of the `EMAs` and `lockMode` fields, thereby compromising the integrity of the `OraclePack` data.

```
uint256 internal constant UPPER_118BITS_MASK =
    0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFC00000000000000000000000000000;

function rebaseOraclePack(
    OraclePack oraclePack
) internal pure returns (int24 _newReferenceTick, OraclePack rebasedOraclePack) {
    unchecked {
        int24 _referenceTick = oraclePack.referenceTick();

        _newReferenceTick = getMedianTick(oraclePack);
        int24 deltaOffset = _newReferenceTick - _referenceTick;

        uint256 _newResiduals;
        for (uint8 i; i < 8; ++i) {
            int24 _residual = oraclePack.residualTick(i);
            int24 newEntry = _residual - deltaOffset;
            _newResiduals += (uint256(uint16(uint24(newEntry) & 0x0FFF)) & 0x0FFF) << (i * 12);
        }
    }
}
```

```
@>         rebasedOraclePack = OraclePack.wrap(
            (OraclePack.unwrap(oraclePack) & UPPER_118BITS_MASK) +
            (uint256(uint24(_newReferenceTick) & BITMASK_UINT22) << 96) +
            uint96(_newResiduals)
        );
    }
}
```

The portion **F** is only 128 bits long, and when combined with **C** (2 bits), it results in a total of 130 bits. During the **&** calculation, this leads to the unintended clearing of the lower bits of **EMAs** and **lockMode** in the original data.

🔗 Recommended mitigation steps

Increase the length of the `UPPER_118BITS_MASK` to 138 bits, ensuring that the `EMAs` and `lockMode` fields are not inadvertently cleared.

🔗 Proof of Concept

Please add the following test content to a new file under `test/foundry/core/` (e.g., `test/foundry/core/poc_rebaseOraclePack.t.sol`) and execute it:

[View detailed Proof of Concept](#)

Output:

```
Ran 1 test for test/foundry/core/poc_rebaseOraclePack.t.sol:Poc
[PASS] test_poc_rebaseOraclePack() (gas: 2713130)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 34.70s (420.68ms CPU time)
```

Panoptic mitigated:

Correct `UPPER_138BITS_MASK` to preserve `lockMode` and `EMAs`.

Status: Mitigation confirmed. Full details in reports from [Valves](#), [edoscoba](#), and [Nyx](#).

🔗 [M-05] Division-by-zero in long-leg collateral requirement can block solvency checks and `dispatchFrom` (liquidation/force-exercise) for `tickSpacing==1` pools

Submitted by [niffylord](#), also found by [Oxnija](#), [Agontuk](#), [Blackdruid](#), [Diavolo](#), [edoscoba](#), [Ituba](#), [legat](#), [LimeiBBQ](#), and [VulSight](#)

For pools with `tickSpacing == 1`, a 1-tick-wide long leg (`width == 1`) evaluated at `atTick == strike` triggers a division-by-zero revert inside `RiskEngine` margin computation. Because `PanopticPool.dispatchFrom(...)` always includes `currentTick` in its tick set, any liquidation/force-exercise/settle action that hits `currentTick == strike` can hard-revert, potentially blocking third-party resolution actions at specific prices.

🔗 Exploit narrative (attack steps)

Assumptions:

- Target pool has `tickSpacing == 1`.
- Attacker can obtain/hold a long position with a 1-tick-wide leg and strike `T`.

Steps:

1. Ensure the position exists with `width == 1`, `tickSpacing == 1`, `strike == T`.
2. When `dispatchFrom(...)` evaluates solvency at ticks including `currentTick`, and `currentTick == T`, margin evaluation calls into `RiskEngine` and reverts (division-by-zero), blocking the action.

🔗 Affected components/code references (GitHub links)

- Division-by-zero source (long-leg branch): `RiskEngine.sol` [L1508](#) and `RiskEngine.sol` [L1541](#)
- `dispatchFrom` includes `currentTick` in tick set: `PanopticPool.sol` [L1360](#) and `PanopticPool.sol` [L1392](#)

🔗 Proof of Concept

This PoC uses the existing V3 harness and demonstrates liquidation reverting with a division-by-zero error.

Put this function inside `PanopticPoolTest` in: `test/foundry/coreV3/PanopticPool.t.sol`

Run:

```
FOUNDRY_PROFILE=ci_test FOUNDRY_ETH_RPC_URL=https://ethereum.publicnode.com \
forge test \
--match-path test/foundry/coreV3/PanopticPool.t.sol \
--match-test test_poc_WidthOneLongAtStrike_DivisionByZero_BlocksLiquidation
```

[View detailed Proof of Concept](#)

- When `tickSpacing == 1` and `width == 1`, the code path sets `positionWidth = tickUpper - tickLower = 1`.
- In the long-leg branch, `distanceFromStrike = max(positionWidth / 2, abs(atTick - strike))` becomes `max(0, 0) = 0` when `atTick == strike`.
- The subsequent computation divides by `(distanceFromStrike * expValue)`, which becomes a division-by-zero revert. The PoC triggers this exact state and asserts `stdError.divisionError` during liquidation.

🔗 Recommended mitigation steps

Add a `distanceFromStrike == 0` guard in the long-leg branch (e.g., clamp to 1), and/or disallow `width==1` longs for `tickSpacing==1` pools if that's an acceptable constraint.

[Panoptic mitigated:](#)

No more division-by-zero in long-leg collateral requirement.

Status: Mitigation confirmed. Full details in reports from [Valves](#), [edoscoba](#), and [Nyx](#).

🔗 [M-06] `dispatchFrom()` liveness DoS via `StaleOracle` : spot price manipulation blocks liquidations, force exercises, and premium settlements

Submitted by [fuzious](#), also found by [brotzumax](#), [LinKenji](#), and [Valves](#)

`PanopticPool.sol` [L1388-L1389](#)

🔗 Finding description

The `PanopticPool.dispatchFrom()` function, which serves as the shared entrypoint for liquidations, force

exercises, and long premium settlements, unconditionally reverts with `Errors.StaleOracle()` when the live spot tick (`currentTick`) deviates more than `tickDeltaLiquidation` ticks from the protocol's internal TWAP (`twapTick`).

```
1389
1390 unchecked {
1391     (RiskParameters riskParameters, ) = getRiskParameters(0);
1392     int256 MAX_TWAP_DELTA_LIQUIDATION = int256(
1393         uint256(riskParameters.tickDeltaLiquidation())
1394     );
1395     if (Math.abs(currentTick - twapTick) > MAX_TWAP_DELTA_LIQUIDATION)
1396         revert Errors.StaleOracle();
1397 }
```

The default threshold is `MAX_TWAP_DELTA_LIQUIDATION = 513` ticks (defined in `RiskEngine.sol` [L76](#)), corresponding to approximately a 5.27% price deviation ($1.0001^{513} \approx 1.0527$).

This gate executes **before** the function determines whether the operation is:

- A liquidation (`solvent == 0`)
- A force exercise (`solvent == numberOfTicks && toLength == finalLength + 1`)
- A long premium settlement (`solvent == numberOfTicks && toLength == finalLength`)

Consequently, when `abs(currentTick - twapTick) > tickDeltaLiquidation`, all `dispatchFrom` operations are blocked for all callers.

Impact

1. **Delayed/Blocked liquidations:** Insolvent positions remain open longer than intended. In stressed markets, this increases the probability that losses socialize to PLPs (Panoptic Liquidity Providers) or manifest as bad debt.
2. **Blocked force exercises:** Legitimate force exercise operations on exercisable long positions are prevented.
3. **Blocked premium settlements:** Long premium settlement cannot occur, preventing sellers from collecting owed premium.
4. **Liquidator griefing:** Liquidators can be grieved into repeated reverted transactions (gas loss) if they do not utilize private orderflow.
5. **Economic exploitation:** An insolvent user (or MEV searcher acting on their behalf) can front-run detected liquidation transactions with a swap that pushes `currentTick` beyond the threshold, then optionally back-run to restore the price, repeatedly delaying liquidation indefinitely.

Attack scenarios

Scenario A: Mempool-reactive liquidation DoS

1. Account becomes insolvent; a liquidator submits a liquidation via `dispatchFrom`.
2. Account owner (or MEV searcher) observes the transaction in the public mempool.
3. Attacker front-runs with a swap that pushes `currentTick` beyond $\pm$ `tickDeltaLiquidation` ticks from `twapTick`.
4. Liquidation executes and reverts with `Errors.StaleOracle()`.
5. Attacker can optionally back-run to restore price and minimize exposure.
6. Repeat as needed to delay liquidation indefinitely.

Scenario B: Liveness failure during volatility

Even without a strategic attacker, large market moves can naturally place the spot price far from the protocol's internal TWAP/EMA state. During that window, `dispatchFrom` is unavailable, coinciding with the

exact periods where liquidations are most critically needed.

🔗 Recommended Mitigation Steps

The `StaleOracle` gate exists to prevent liquidation paths from executing at a potentially manipulated spot price. Removing it outright reintroduces price-manipulation attacks against liquidation pricing and accounting. Consider the following alternatives:

1. Two-step liquidation (Mark and Execute)

- Mark insolvency using TWAP/median-based criteria without reverting on divergence.
- Execute liquidation after a delay or once oracle state catches up.

2. Degraded mode: Proceed using oracle ticks only

- If `abs(currentTick - twapTick) > threshold`, proceed but substitute `currentTick` with `twapTick` (or `medianTick / latestTick`) for all liquidation/settlement pricing inputs.

3. Rate-limited/Governance-guarded bypass

- Allow a privileged actor (or tightly scoped guardian role) to bypass the gate for emergency liquidation of specific accounts/pools.

🔗 Proof of Concept

PoC demonstrates end-to-end that:

1. A valid liquidation succeeds when spot price is within the `tickDeltaLiquidation` threshold.
2. The same valid liquidation reverts with `Errors.StaleOracle` when spot is moved out-of-range.
3. Force exercise and premium settlement are similarly blocked when out-of-range.

Test Coverage

Test	What it proves
<code>test_PoC_LiquidationExecutes_WhenInsolvent_AndPriceInRange</code>	Valid liquidation executes when <code>abs(currentTick - twapTick) <= tickDeltaLiquidation</code>
<code>test_PoC_LiquidationDoS_BlocksValidLiquidation_WhenPriceOutOfRange</code>	Same valid liquidation reverts with <code>Errors.StaleOracle</code> when spot is out-of-range
<code>test_PoC_ForceExerciseExecutes_WhenPriceInRange</code>	Valid force exercise executes in-range (emits <code>ForcedExercised</code>)
<code>test_PoC_ForceExerciseDoS_BlocksValidForceExercise_WhenPriceOutOfRange</code>	Force exercise blocked by <code>Errors.StaleOracle</code> when out-of-range
<code>test_PoC_PremiumSettlementExecutes_WhenPriceInRange</code>	Premium settlement executes in-range (emits <code>PremiumSettled</code>)
<code>test_PoC_PremiumSettlementDoS_BlocksValidSettlement_WhenPriceOutOfRange</code>	Settlement blocked by <code>Errors.StaleOracle</code> when out-of-range
<code>test_PoC_StaleOracleBoundary_StrictGreaterThan_WhenReachable</code>	Gate is strict <code>></code> (not <code>>=</code>) at exact boundary

🔗 PoC Diff

To apply the PoC, create the file `test/foundry/core/LiquidationDoSPoC.t.sol` with the following content:

[View detailed Proof of Concept](#)

[Panoptic mitigated:](#)

No more DoS via `StaleOracle` for liquidations.

Status: Mitigation confirmed. Full details in report from [Nyx](#).

🔗 [M-07] Liquidator can receive an inflated bonus against PLPs on

`PanopticPool._liquidate`

Submitted by [hecker_trieu_tien](#), also found by [Albert](#)

`CollateralTracker.sol` [L1595](#)

🔗 Summary

`CollateralTracker.settleBurn` computes `tokenPaid` without commission/interest, which will cause a higher reported `netPaid` for PLPs as a liquidator will use the understated `netPaid` to receive a larger liquidation bonus than the liquidatee actually paid.

🔗 Root Cause

In `contracts/CollateralTracker.sol`, `settleBurn` returns `tokenPaid` derived from `_updateBalancesAndSettle` before commission shares are burned, so the amount returned to the pool does not include the commission paid by the liquidatee.

- `_updateBalancesAndSettle` computes `tokenToPay` from AMM delta, net borrows, and realized premium, then returns it as `tokenPaid`.
- `settleBurn` then burns commission shares based on `realizedPremium`, but the return value is not adjusted to reflect that additional cost.
- `PanopticPool._liquidate` uses the unadjusted `netPaid` to compute the liquidation bonus.

🔗 Finding description

Step 1: `settleBurn` returns a `tokenPaid` value that does not include commission burned after the fact.

```
// contracts/CollateralTracker.sol
function settleBurn(...) external onlyPanopticPool returns (int128) {
    (, int128 tokenPaid, uint256 _totalAssets, uint256 _totalSupply) = _updateBalancesAndSettle(
        optionOwner,
        false,
        longAmount,
        shortAmount,
        ammDeltaAmount,
        realizedPremium
    );

    if (realizedPremium != 0) {
        uint128 commissionFee = ...;
        uint256 sharesToBurn = Math.mulDivRoundingUp(commissionFee, _totalSupply, _totalAssets);
        _burn(optionOwner, sharesToBurn);
    }

    return tokenPaid;
}
```

`tokenPaid` is computed before commission is burned, so it under-reports total value paid by the liquidatee in this burn.

Step 2: `_burnAllOptionsFrom` aggregates `tokenPaid` into `netPaid` without commission.

```
// contracts/PanopticPool.sol
(paidAmounts, premiasByLeg[i], ) = _burnOptions(...);
netPaid = netPaid.add(paidAmounts);
```

`paidAmounts` is built from `settleBurn` return values, so the omission propagates to `netPaid`.

Step 3: `_liquidate` uses `netPaid` to compute the liquidation bonus.

```
// contracts/PanopticPool.sol
(netPaid, premiasByLeg) = _burnAllOptionsFrom(...);
(bonusAmounts, collateralRemaining) = riskEngine().getLiquidationBonus(
    tokenData0,
    tokenData1,
    Math.getSqrtRatioAtTick(twapTick),
    netPaid,
    shortPremium
);
```

The bonus is calculated using a value that excludes commission paid, which inflates the liquidatee's remaining collateral and therefore the bonus.

🔗 Impact

Liquidation bonuses can be slightly overstated relative to the liquidatee's true collateral change, creating a small protocol loss when commissions are non-zero.

🔗 Recommended mitigation steps

Include the commission fee (in asset terms) in the `tokenPaid` returned from `settleBurn`, or return the commission separately and incorporate it into `netPaid` before calling `getLiquidationBonus`.

Sample mitigation (adjust return value to include commission):

```
// contracts/CollateralTracker.sol
function settleBurn(...) external onlyPanopticPool returns (int128) {
    (, int128 tokenPaid, uint256 _totalAssets, uint256 _totalSupply) = _updateBalancesAndSettle(
        optionOwner,
        false,
        longAmount,
        shortAmount,
        ammDeltaAmount,
        realizedPremium
    );

    if (realizedPremium != 0) {
        uint128 commissionFee = ...;
        uint256 sharesToBurn = Math.mulDivRoundingUp(commissionFee, _totalSupply, _totalAssets);
        _burn(optionOwner, sharesToBurn);
        tokenPaid += int128(uint128(commissionFee));
    }

    return tokenPaid;
}
```

🔗 Proof of Concept

`test/foundry/core/PanopticPool.t.sol`

[View detailed Proof of Concept](#)

[Panoptic mitigated:](#)

Include commissions in `tokenPaid`.

Status: Mitigation confirmed. Full details in reports from [Valves](#), [edoscoba](#), and [Nyx](#).

🔗 **[M-08] Wide-range short legs can revert solvency checks and block liquidations (`Errors.InvalidTick`)**

Submitted by [edoscoba](#)

- RiskEngine.sol [L1489-L1505](#)
- Math.sol [L187-L245](#)
- PanopticPool.sol [L1793-L1803](#)

🔗 Finding description and impact

In `RiskEngine._getRequiredCollateralSingleLegNoPartner`, short legs use an “in-range interpolation” path when `atTick` is within `[tickLower, tickUpper]`. That path computes: `scaleFactor = Math.getSqrtRatioAtTick(tickUpper - tickLower)`

However, `getSqrtRatioAtTick` only supports ticks within `[-887_272, 887_272]`. While `tickLower` and `tickUpper` themselves are validated to be within bounds, their *difference* can exceed `887_272` for wide ranges, because the range width is `width * tickSpacing` (with `width` up to `4095` and Uniswap v4 allowing large `tickSpacing` values). In that case, any solvency computation that evaluates the position while the price is in-range will revert with `Errors.InvalidTick()`.

This is exploitable as a denial-of-service on liquidation and other solvency-gated operations:

- An attacker can create a short position with `tickUpper - tickLower > 887_272` while ensuring the current tick is initially outside the range (so the in-range branch is not executed during that check).
- If/when the market tick later enters the range, solvency checks for that account revert deterministically.
- Liquidation paths that require solvency evaluation can be bricked, enabling “insolvent-but-unliquidatable” accounts and increasing the chance of protocol bad debt.
- The same revert can also DoS collateral-withdraw eligibility checks or any other path that calls `riskEngine.isAccountSolvent(...)` at an in-range tick.

This issue is not listed in `README.md` “Publicly known issues” and does not appear in `2025_12_panoptic_v12_findings.md`.

🔗 Recommended mitigation steps

- Replace `getSqrtRatioAtTick(tickUpper - tickLower)` with a computation that never requires an out-of-domain tick input, e.g. derive the ratio from individually valid endpoints:
 - `sqrtLower = Math.getSqrtRatioAtTick(tickLower)`
 - `sqrtUpper = Math.getSqrtRatioAtTick(tickUpper)`
 - `scaleFactor = sqrtUpper * FP96 / sqrtLower` (choose rounding direction to preserve conservative collateral requirements)
- Avoid “clamping” `(tickUpper - tickLower)` as a primary fix unless you can prove the result remains conservative; clamping changes the intended economics and can understate requirements.
- Optionally, add a mint-time validation that rejects short legs with `tickUpper - tickLower > Constants.MAX_POOL_TICK` to fail fast, but the RiskEngine-side fix should still be implemented to prevent protocol-level reverts.

🔗 Proof of Concept

A runnable PoC is included as a Foundry test:

```
test/foundry/core/WideRangeShortInRangeSqrtRatioRevertPoC.t.sol
```

Run: `forge test --match-contract WideRangeShortInRangeSqrtRatioRevertPoC -vvv`

The PoC shows the same wide-range short leg:

1. Returns a valid collateral requirement when evaluated out-of-range, and
2. Reverts with `Errors.InvalidTick()` when evaluated at a tick inside the range (triggering the in-range

interpolation branch).

[View detailed Proof of Concept](#)

Panoptic marked as informative.

[\[M-09\] Incorrect collateral calculation for delayed swap strategies](#)

Submitted by [eternal1328](#), also found by [astra39100](#), [Coachmike](#), [Funen](#), [lamassu](#), [prk0](#), and [YZXO](#)

RiskEngine.sol [L2018-L2038](#)

[Finding description](#)

The `_computeDelayedSwap` function in the RiskEngine contract incorrectly calculates collateral requirements for Delayed Swap strategies. Instead of netting the credit leg value against the loan requirement (which would provide capital efficiency for hedged positions), it uses `max (loanRequirement, creditValue)`. This treats the credit position as a liability floor rather than a deduction, causing artificially inflated collateral requirements. When the credit value exceeds the loan requirement, the collateral requirement becomes the full credit value instead of being reduced to a minimal floor.

[Root Cause](#)

The root cause is flawed financial logic in the `_computeDelayedSwap` function. For a hedged delayed swap position (pairing a loan leg with a credit leg), the correct economic treatment should be to subtract the credit value from the loan requirement, with a minimum floor. Instead, the implementation incorrectly treats the credit as an alternative requirement, taking the maximum of the two values. This violates the fundamental principle of netting in risk calculation for offsetting positions.

Original code block has been omitted. [View complete submission.](#)

The vulnerable code computes both the loan requirement and the converted credit value, then returns the maximum of the two values. This is incorrect financial logic for a delayed swap where the credit leg should offset the loan requirement. The correct approach would be to subtract the converted credit from the loan requirement (with a minimum floor), providing capital efficiency for the hedged position. The current implementation penalizes users by requiring collateral equal to the full credit value when it exceeds the loan requirement.

[Impact](#)

Users with Delayed Swap strategies face artificially inflated collateral requirements, making solvent accounts appear insolvent. This can trigger unfair liquidations where liquidators seize user collateral, causing direct financial loss. The impact is HIGH severity because it leads to direct asset loss through incorrect solvency determinations. The vulnerability affects all users employing Delayed Swap strategies, potentially causing widespread liquidation events and loss of user funds.

[Proof of Concept](#)

Runnable PoC (Foundry test):

- PoC file (custom, entire file included): `2025-12-panoptic/test/foundry/poc/H017_DelayedSwap_MaxNotNetting.t.sol`
- Test: `test_POC_H017_delayedSwap_requirementUsesMax_notNetting`

How to run:

```
cd 2025-12-panoptic

anvil --silent --port 8545 &

forge test --fork-url http://127.0.0.1:8545 \
  --match-path test/foundry/poc/H017_DelayedSwap_MaxNotNetting.t.sol \
  --match-test test_POC_H017
```

PoC source (full file):

[View detailed Proof of Concept](#)

PoC shared dependency:

File: `2025-12-panoptic/test/foundry/poc/Utils/PoCUtils.sol`

[View detailed Proof of Concept](#)

[Panoptic mitigated:](#)

Netting the credit against the loan legs.

Status: Mitigation confirmed. Full details in reports from [Valves](#), [edoscoba](#), and [Nyx](#).

[\[M-10\] PLPs can withdraw assets needed by long positions, temporarily locking buyers](#)

Submitted by [Henri](#)

`CollateralTracker.sol` [L65](#)

[Summary](#)

LPs can withdraw all available liquidity from a collateral contract, making it impossible for long positions to be closed; neither through `burnOptions` nor `forceExercise`.

[Finding description](#)

The `maxWithdraw` function allows LPs to withdraw up to `s_depositedAssets` - 1:

```
function maxWithdraw(address owner) public view returns (uint256 maxAssets) {
    uint256 depositedAssets = s_depositedAssets;
    unchecked {
        uint256 available = depositedAssets > 0 ? depositedAssets - 1 : 0;
        uint256 balance = convertToAssets(balanceOf[owner]);
        return panopticPool().numberOfLegs(owner) == 0 ? Math.min(available, balance) : 0;
    }
}
```

When a long position is created, tokens are pulled from the AMM back into the Panoptic Pool. These tokens increase `s_depositedAssets`, but they are not reserved - LPs can freely withdraw them.

The problem occurs when an LP withdraws enough assets that there isn't sufficient liquidity left for longs to return their borrowed amount to the AMM when closing. This is more likely to happen when shorts and long are quite unbalanced (high utilisation), then `s_depositedAssets` would be low and easier for a LP to deplete the pool.

`forceExercise` cannot really help here either:

- It can only target positions with at least one long leg, so pure short positions cannot be force-exercised to replenish the pool.
- A mixed position with a delta more short than long could be closed but only the delta would replenish the pool. It is not sure that there would be enough of these specific positions.
- This could become costly for the long to close its position.

The current workaround would be for the buyer to deposit the missing amount into the collateral contract before closing the long. This is problematic because buyers shouldn't need extra tokens to close their own position, especially as the borrowed amount can go up to 5 times their collateral.

[Impact](#)

- Long holders cannot close their positions, leading to potential losses if the market moves against them.
- If `netLiquidity` is low on a given chunk, shorts also become unable to close since they would need to `forceExercise` longs.

A whale LP could exploit this by waiting for the right moment to withdraw and trap long holders in their positions.

Example:

1. Bob (LP) deposits 100 ETH in `vault0`
2. Seller deposits 5 ETH in `vault0` as collateral
3. Alice deposits 1 ETH in `vault0` as collateral
4. `s_depositedAssets = 106` ETH
5. Seller shorts a call of 10 ETH on chunk A
6. Alice longs a call of 1 ETH on chunk A
7. `s_depositedAssets = 106 - 10 + 1 = 97` ETH
8. Bob withdraws 97 (+fees) ETH (all available since his `balance > s_depositedAssets`)
9. `s_depositedAssets = 0` ETH
10. Alice tries to close her long, the SPFM needs to return 1 ETH from the `positionManager` to the AMM
→ reverts

Proof of Concept

Add the following test to `test/foundry/core/PanopticPool.t.sol`:

Logs

```
Bound result 1
vault 0 1000000
vault 1 1000000
Seller balance - vault 0 0
Seller balance - vault 1 0
Bob balance - vault 0 0
Bob balance - vault 1 0
Alice balance - vault 0 0
Alice balance - vault 1 0
max Bob can withdraw 0
Bob balance - vault 0 - after deposit 1000000000000000000
Seller balance - vault 0 - after deposit 500000000000000000
Alice balance - vault 0 - after deposit 100000000000000000
vault 0 106000000000000100000
Bound result 1
Bound result 20476
Bound result 1000000000000001
currentTick 195481
tickLower 204760
tickUpper 204770
Seller short positionSize 1000000000000000000
Alice long positionSize 1000000000000000000
max Bob can withdraw 97000000000000999998
```

[View detailed Proof of Concept](#)

[Panoptic mitigated:](#)

Fixes a calculation error where `creditedAssets` was being subtracted from `available`.

Status: Mitigation confirmed. Full details in reports from [Valves](#), [edoscoba](#), and [Nyx](#).

[\[M-11\] An attacker can manipulate oracle easily](#)

Submitted by [Tupaia](#)

- `OraclePack.sol` [L536-L567](#)
- `PanopticPool.sol` [L552-L558](#)

Finding description

This is a critical oracle manipulation vulnerability where any user can call `PanopticPool::pokeOracle()` to update the internal oracle with a manipulatable `currentTick`, and once the oracle is updated in an epoch, it cannot be corrected for the remainder of that 64-second epoch period, allowing attackers to frontrun legitimate oracle updates and gradually manipulate the oracle across multiple epochs using flash loans.

The root cause lies in the design of `OraclePack::computeInternalMedian()` which only updates the oracle pack when the current epoch differs from the recorded epoch. The function checks if enough time has passed by comparing epochs.

```
// File: contracts/types/OraclePack.sol, lines 546-565
uint256 currentEpoch;
bool differentEpoch;
int256 timeDelta;
{
    currentEpoch = (block.timestamp >> 6) & 0xFFFFF; // 64-long epoch, taken mod 2**24
    uint256 recordedEpoch = oraclePack.epoch();
    differentEpoch = currentEpoch != recordedEpoch;
    timeDelta = int256(uint256(uint24(currentEpoch - recordedEpoch))) * 64; // take a rough time delta, b
}
// only proceed if last entry is in a different epoch
if (differentEpoch) {
    int24 clampedTick = clampTick(currentTick, oraclePack, clampDelta);
    _updatedOraclePack = insertObservation(
        oraclePack,
        clampedTick,
        currentEpoch,
        timeDelta,
        EMAperiods
    );
}
```

Once `differentEpoch` is true and the oracle is updated, subsequent calls to `computeInternalMedian()` within the same epoch will return an unchanged oracle pack because `differentEpoch` remains false until the next 64-second epoch boundary. The `PanopticPool::pokeOracle()` function is publicly accessible without any access controls, allowing any user to trigger oracle updates.

```
// File: contracts/PanopticPool.sol, lines 552-558
function pokeOracle() external {
    int24 currentTick = getCurrentTick();

    (, OraclePack oraclePack) = riskEngine().computeInternalMedian(s_oraclePack, currentTick);

    if (OraclePack.unwrap(oraclePack) != 0) s_oraclePack = oraclePack;
}
```

The `currentTick` used in `pokeOracle()` is obtained from `getCurrentTick()`, which reads the current tick from the Uniswap pool. This tick can be manipulated using flash loans to create large price movements. While `clampTick()` limits the maximum change per update to `MAX_CLAMP_DELTA` (149 ticks), this protection only applies within a single update.

```
// File: contracts/types/OraclePack.sol, lines 511-528
function clampTick(
    int24 newTick,
    OraclePack _oraclePack,
    int24 clampDelta
) internal pure returns (int24 clamped) {
```

```

unchecked {
    int24 _lastTick = _oraclePack.lastTick();

    // Clamp lastObservedTick to be within clampDelta of lastTick
    if (newTick > _lastTick + clampDelta) {
        clamped = _lastTick + clampDelta;
    } else if (newTick < _lastTick - clampDelta) {
        clamped = _lastTick - clampDelta;
    } else {
        clamped = newTick;
    }
}
}

```

An attacker can exploit this by frontrunning legitimate `pokeOracle()` calls. When a new epoch begins, the attacker uses a flash loan to manipulate the Uniswap pool price, then calls `pokeOracle()` to update the oracle with the manipulated tick. Since the oracle can only be updated once per epoch, legitimate calls to `pokeOracle()` later in the same epoch will have no effect, leaving the manipulated value in place for the entire 64-second epoch.

The attacker can repeat this process across multiple epochs, gradually moving the oracle in their desired direction by up to 149 ticks per epoch. Over several epochs, this cumulative manipulation can significantly distort the oracle price, affecting all positions that rely on the oracle for pricing and solvency checks.

Impact

Attackers can manipulate the oracle by frontrunning legitimate updates and using flash loans to gradually move the oracle price across multiple epochs, causing all positions to be evaluated at incorrect prices and leading to incorrect solvency validations, unfair liquidations, or allowing insolvent positions to remain active.

Recommended mitigation steps

Modify the oracle mechanism so that the oracle is updated continuously with the current tick as time progresses, similar to Uniswap TWAP, rather than allowing only one update per epoch. This would prevent attackers from locking in manipulated values for an entire epoch period.

```

// File: contracts/types/OraclePack.sol, lines 536-567
function computeInternalMedian(
    OraclePack oraclePack,
    int24 currentTick,
    uint96 EMAperiods,
    int24 clampDelta
) internal view returns (int24 _medianTick, OraclePack _updatedOraclePack) {
    unchecked {
        _medianTick = getMedianTick(oraclePack);

        uint256 currentEpoch;
        bool differentEpoch;
        int256 timeDelta;
        {
            currentEpoch = (block.timestamp >> 6) & 0xFFFFFFF;
            uint256 recordedEpoch = oraclePack.epoch();
            differentEpoch = currentEpoch != recordedEpoch;
            timeDelta = int256(uint256(uint24(currentEpoch - recordedEpoch))) * 64;
            // Calculate actual time delta in seconds
            timeDelta = int256(block.timestamp) - int256(oraclePack.timestamp());
        }
        // only proceed if last entry is in a different epoch
        if (differentEpoch) {
            // Update oracle if enough time has passed (e.g., at least 64 seconds)
            if (timeDelta >= 64) {
                int24 clampedTick = clampTick(currentTick, oraclePack, clampDelta);
                _updatedOraclePack = insertObservation(
                    oraclePack,
                    clampedTick,
                    currentEpoch,
                    timeDelta,
                    EMAperiods
                );
            }
            // Allow multiple updates within the same epoch if enough time has passed
        }
    }
}

```

Alternatively, restrict `pokeOracle()` to only be callable by trusted addresses or implement a commit-reveal scheme to prevent frontrunning.

🔗 Proof of Concept

The following test demonstrates the vulnerability by showing that an attacker can manipulate the oracle by frontrunning `pokeOracle()` calls and gradually moving the oracle across multiple epochs, with the manipulated value locked in for the entire 64-second epoch period.

To run this PoC, add the following test function to `test/foundry/core/Misc.t.sol`:

[View detailed Proof of Concept](#)

This test can be run with `forge test --match-test test_PoC_OracleManipulation -vvv` to demonstrate the vulnerability. The test shows that:

1. An attacker can manipulate the pool price using swaps (simulating flash loans) and call `pokeOracle()` to update the oracle.
2. Once updated in an epoch, the oracle cannot be corrected for the remainder of that 64-second epoch, even if a legitimate user attempts to update it with the correct price.
3. The attacker can repeat this process across multiple epochs to gradually manipulate the oracle by up to 149 ticks per epoch (limited by `MAX_CLAMP_DELTA`).
4. The cumulative manipulation affects all positions that rely on the oracle for pricing, causing incorrect solvency validations and potentially unfair liquidations.

Panoptic disputed

🔗 [M-12] Self-settlement via `dispatchFrom` bypasses refund mechanism allowing underfunded debt settlement

Submitted by [merlinboji](#), also found by [AnantaDeva](#) and [Valves](#)

- `PanopticPool.sol` [L1671-L1703](#)
- `PanopticPool.sol` [L1598-L1664](#)

🔗 Finding description

The `PanopticPool._settlePremium()` and `PanopticPool._forceExercise()` allow position owners to act as the caller (`account == msg.sender`) via `dispatchFrom`. However, these flows rely on a refund mechanism that assumes the caller is a distinct third party who can cover any post-settlement shortage.

When `account == msg.sender`, this assumption breaks because:

1. `CollateralTracker.delegate()` inflates the user's balance with phantom shares (`~type(uint248).max`), bypassing balance checks during settlement.
2. `CollateralTracker.refund()` becomes a self-transfer (`refund(account, account, ...)`) and injects no real value.
3. Settlement proceeds even when `refundAmounts != 0`, meaning a shortage exists that is never actually covered.

This allows settlement to succeed even when the user lacks sufficient real balance to fully cover the debt (interest + premium).

The refund mechanism is designed such that if the target account lacks `token0` or `token1`, the caller supplies the shortage token in exchange for receiving the other token from the target account. The `CollateralTracker.refund()` explicitly states this assumption:

`CollateralTracker.sol` [L1369-L1382](#)

```

    ///...
    /// @dev Assumes that the refunder has enough money to pay for the refund.
    ///...
    function refund(address refunder, address refundee, int256 assets) external onlyPanopticPool {
        if (assets > 0) {
            _transferFrom(refunder, refundee, convertToShares(uint256(assets)));
        } else {
            uint256 sharesToTransfer = convertToShares(uint256(-assets));
            if (balanceOf[refundee] < sharesToTransfer)
                revert Errors.NotEnoughTokens(
                    address(this),
                    uint256(-assets),
                    convertToAssets(balanceOf[refundee])
                );
            _transferFrom(refundee, refunder, sharesToTransfer);
        }
    }
}

```

This assumption fails when `refunder == refundee` because a self-transfer cannot inject real value to cover the shortage.

During self-settlement, the flow executes as:

```
delegate() -> settlement -> refund() -> revoke()
```

Consider the following example:

Let:

- `B` = user's real balance (shares)
- `I` = phantom shares added by delegate ($\sim 2^{248}$)
- `X` = total shares the settlement attempts to burn
- `s = X - B` = shortage (`s > 0`)

During delegation:

```
balance = B + I (assume B > interestShares)
```

Settlement burns X shares successfully: Since balance is inflated.

After burn (before revoke): Phantom shares were consumed.

```
balance = (B + I) - X = I - s
```

Revoke repair: Because phantom was consumed.

```
_internalSupply += s
balanceOf[account] = 0
```

Net supply change:

```
 $\Delta TS = -X + s = -(B + s) + s = -B$ 
```

The contract extracts only the user's real balance `B`, even though settlement attempted to burn `X` (where `X > B`). The remaining shortage `s` was never funded.

Recommended mitigation steps

Update `_settlePremium` and `_forceExercise` to handle self settlement cases so that when `account == msg.sender`, settlement cannot proceed if the account's real balance is insufficient to cover the required burn amount.

[Panoptic mitigated:](#)

Refactored part of the `delegate` / `revoke` workflow.

Status: Mitigation confirmed. Full details in reports from [Valves](#), [edoscoba](#), and [Nyx](#).

[\[M-13\] Intra-epoch `rateAtTarget` updates in `CollateralTracker.\_updateInterestRate\(\)` allow compounding interest rate errors](#)

Submitted by [merlinboji](#), also found by [Dulgiq](#)

- `RiskEngine.sol` [L2203](#)
- `RiskEngine.sol` [L2218-L2221](#)
- `CollateralTracker.sol` [L1047-L1054](#)
- `CollateralTracker.sol` [L970-L976](#)

Finding description

The `CollateralTracker._accrueInterest()` incorrectly updates `s_marketState.rateAtTarget()` on every call, even when multiple operations occur within the same epoch. This creates a compounding error where the interest rate calculation uses incorrect time references.

The protocol uses epochs of 4 seconds (`block.timestamp >> 2`) to batch interest calculations. However, there's a critical mismatch in how `rateAtTarget` and `marketEpoch` are updated:

When `block.timestamp` falls between epoch boundaries, the `elapsed` time calculation in `RiskEngine._borrowRate()` becomes non-zero even though the epoch hasn't advanced:

```
→ uint256 block_timestamp = 1767776587
→ uint256 currentEpoch = block_timestamp >> 2
→ uint256 previousTime = currentEpoch << 2
→ previousTime
Type: uint256
└ Hex: 0x695e2148
└ Hex (full word): 0x00000000000000000000000000000000000000000000000000000000000000695e2148
└ Decimal: 1767776584
→ block_timestamp - previousTime <-- elapsed time
Type: uint256
└ Hex: 0x3
└ Hex (full word): 0x0000000000000000000000000000000000000000000000000000000000000003
└ Decimal: 3 <-- elapsed time
```

The issue can be summarized as follows:

1. The `s_marketState.rateAtTarget()` gets updated immediately in `CollateralTracker._updateInterestRate()` and then update again in `CollateralTracker._accrueInterest()`.
2. However, `s_marketState.marketEpoch()` is only updated when `deltaTime > 0` in `CollateralTracker._calculateCurrentInterestState()`.
3. Within the same epoch (`currentEpoch == previousEpoch`), `deltaTime = 0`, so the epoch doesn't advance.
4. But `rateAtTarget` still updates based on the non-zero elapsed calculation.

`RiskEngine.sol` [L2187-L2255](#)

Original code block has been omitted. [View complete submission](#).

🔗 Impact

The `borrowIndex` becomes manipulated through repeated `rateAtTarget` updates within the same epoch. This index determines user interest payments and position solvency, leading to incorrect debt calculations.

Each call uses the previously manipulated `rateAtTarget` as the starting point, causing compounding errors. An attacker can trigger multiple calls within a 4-second window to amplify the effect. (This can happen through unintentional manipulation as well, simply by triggering `CollateralTracker._accrueInterest()` multiple times within the same block).

Although the `elapsed` time is capped at less than 4 seconds, which may appear insignificant, the impact is magnified because the rate calculation also depends on utilization. In a single transaction, the highest utilization value is used, allowing this mechanism to be exploited for a greater effect. (See the proof of concept: `test_audit_accrueInterest_manipulateRateAtTarget_combineAttack`.)

```
function _updateInterestRate() internal returns (uint128) {
    (uint128 avgRate, uint256 endRateAtTarget) = riskEngine().updateInterestRate(
    @> _poolUtilizationWad(),
        s_marketState
    );
    s_marketState = s_marketState.updateRateAtTarget(uint40(endRateAtTarget));
    return avgRate;
}
```

🔗 Recommended mitigation steps

Update `rateAtTarget` only if there is actual time passed. If the rate has already been updated within the current epoch, the update should be skipped.

🔗 Proof of Concept

Setup:

- Put the snippet below into the protocol test suite:
`test/foundry/core/CollateralTracker.t.sol:CollateralTrackerTest`
- Run test:
 - `forge test --mt test_audit_accrueInterest_manipulateRateAtTarget_stateProof -vvv`
 - `forge test --mt test_audit_accrueInterest_manipulateRateAtTarget_combineAttack -vvv`
- Note: The helper functions need to be added for the PoC to run (attached in the `The full coded PoC` section)

Results of running the PoC:

[View detailed Proof of Concept](#)

[Panoptic mitigated:](#)

Compute elapsed-time for IRM from epoch delta.

Status: Mitigation confirmed. Full details in reports from [Valves](#), [edoscoba](#), and [Nyx](#).

🔗 [M-14] State-price mismatch in liquidation

Submitted by [devdragon](#), also found by [merlinboii](#)

In `PanopticPool._liquidate`, the protocol evaluates the liquidatee's collateral and debt to determine the liquidation bonus and potential protocol loss. However, there is a fundamental accounting mismatch in how these values are calculated:

1. Stale asset valuation:

When a liquidator calls `PanopticPool.liquidate`, the protocol immediately checks the liquidatee's margin.

```
contracts/PanopticPool.sol

1482:     function _liquidate(
...
1503:         (tokenData0, tokenData1, ) = riskEngine().getMargin(
...
1510:         collateralToken0(),
1511:         collateralToken1()
1512:     );
```

Inside `getMargin`, the `RiskEngine` calls `assetsAndInterest(user)` on the `CollateralTracker`.

```
contracts/CollateralTracker.sol

1091:     function assetsAndInterest(address owner) external view returns (uint256, uint256) {
1092:         return (convertToAssets(balanceOf[owner]), _owedInterest(owner));
1093:     }
```

`CollateralTracker.assetsAndInterest` calculates the user's collateral balance using `convertToAssets`. This function relies on `totalAssets()`, which is defined as:

```
return uint256(s_depositedAssets) + s_assetsInAMM + s_marketState.unrealizedInterest();
```

The `unrealizedInterest` here is a stored value that is only updated during state-modifying `_accrueInterest` calls. If several blocks have passed since the last interaction, this value is significantly lower than the actual interest accrued by the pool.

2. Fresh Debt Valuation:

In the same call, `assetsAndInterest` retrieves the user's debt via `_owedInterest`. Unlike the asset calculation, this function simulates the global borrow index up to the **current block timestamp**:

```
(uint128 currentBorrowIndex, , ) = _calculateCurrentInterestState(
    s_assetsInAMM,
    _interestRateView(_poolUtilizationWadView())
);
```

This ensures the debt is “Price-Fresh.”

3. The Mismatch:

Because the debt is updated to the current second but the pool's assets (which include that debt) are not, the user's share of the pool is undervalued. Essentially, the user is charged the full interest for the elapsed time, but they do not receive their proportional “credit” as a pool participant for that same interest in their collateral valuation.

🔗 Impact

This mismatch systematically underestimates the liquidatee's net worth. In a liquidation:

- **Overestimated protocol loss:** The protocol will believe the liquidatee is deeper in debt than they truly are. This directly triggers `haircutPremia`, where the protocol revokes premiums from honest sellers to cover a “loss” that doesn't actually exist.
- **Systemic disincentive:** Liquidators receive bonuses based on the available collateral. If the collateral is undervalued, liquidators are underpaid, which can lead to delayed liquidations during high-volatility events.

🔗 Recommended mitigation steps

Call `accrueInterest(liquidatee)` at the beginning of the `_liquidate` function in `PanopticPool.sol`. This ensures that both the global pool assets and the liquidatee's specific balance are updated and synchronized

before any valuation occurs.

🔗 Proof of Concept

Create PoC test file `test/foundry/audit/LiquidationMismatch.t.sol` and run `forge test --match-path test/foundry/audit/LiquidationMismatch.t.sol -vvv`:

[View detailed Proof of Concept](#)

Console output:

```
Logs:
Valuation BEFORE Accrual:
  Assets (Stale Price): 5000000000000000000
  Interest (Fresh Debt): 1434795130794140900
  Net Position Value: 48565204869205859100

Valuation AFTER Accrual:
  Assets (Fresh Price): 48612850467710481715
  Interest (Fresh Debt): 0
  Net Position Value: 48612850467710481715

[!] BUG CONFIRMED: Mismatch hidden value of: 47645598504622615
This discrepancy leads to overstated protocol losses and underpaid liquidators.
```

Panoptic commented:

We made the decision to maintain that protocol design choice (ie. the liquidator needs to call `collateralToken accrueInterest()` manually to get the max possible bonus) and did not fix it.

🔗 [M-15] Commission share-burn distribution is JIT-capturable when `builderCode == 0` (default).

Submitted by [fuzious](#), also found by [kindOdev](#) and [Valves](#)

- `CollateralTracker.sol` [L1558-L1561](#)
- `CollateralTracker.sol` [L1637-L1640](#)
- `RiskEngine.sol` [L253-L264](#)
- `PanopticPool.sol` [L839](#)
- `PanopticPool.sol` [L1634](#)

🔗 Summary

When `builderCode == 0` (i.e., `feeRecipient == address(0)`), commissions in `CollateralTracker.settleMint()` / `CollateralTracker.settleBurn()` are implemented as a **share burn from the option owner** without removing assets. This reduces `totalSupply()` while keeping `totalAssets()` roughly constant, immediately increasing `assetsPerShare` for **whoever holds shares at that instant** (including same-block entrants). MEV searchers can JIT-deposit immediately before these commission-burn events to capture the majority of the value.

A stronger implication than “LP yield leakage” also exists: **fee self-rebating / fee bypass**. If a fee payer (or flash-funded actor in an active-trigger flow) can temporarily dominate the share supply at the moment their commission is burned, they can reclaim most of the fee back via the share-price uplift, undermining the commission’s intended economic purpose.

🔗 Finding description and impact

Panoptic distributes some value to shareholders by *burning shares* (reducing `totalSupply()` without removing assets), which increases `assetsPerShare` immediately for whoever holds shares at that moment. This is inherently **Just-In-Time (JIT) liquidity-capturable**: an MEV searcher can temporarily deposit to

become a dominant shareholder right before a burn event, then unwind after, capturing most of the distribution that was economically intended for incumbent LPs.

Root cause mechanics

The share price is defined by ERC4626-style conversions:

- `assetsPerShare` $\approx$ `totalAssets()` / `totalSupply()`
- `totalAssets()` includes tracked pool assets, assets in AMM, and unrealized interest
- `totalSupply()` is `_internalSupply + s_creditedShares`

Any event that burns shares but does **not** remove assets causes:

- `totalSupply()` ↓ (burn)
- `totalAssets()` $\approx$ constant
- `assetsPerShare` ↑ instantly

That instant uplift is captured pro-rata by *whoever is a shareholder at that exact moment*.

Why `builderCode == 0` is not “rare misconfiguration”:

In `CollateralTracker.settleMint()`:

```
// CollateralTracker.sol:1558-1561
if (riskParameters.feeRecipient() == 0) {
    _burn(optionOwner, sharesToBurn);
    emit CommissionPaid(optionOwner, address(0), commissionFee, 0);
}
```

Critically, `feeRecipient == 0` is produced by the DEFAULT builder flow:

- `RiskEngine.getRiskParameters()` computes `feeRecipient` from `builderCode`
- `_computeBuilderWallet(builderCode)` returns `address(0)` if `builderCode == 0`

More importantly, several critical protocol maintenance paths **HARDCODE** `getRiskParameters(0)`:

- `_burnAllOptionsFrom(...)` (used by liquidations): `(RiskParameters riskParameters, ) = getRiskParameters(0);` (Line 839)
- `_forceExercise(...)`: `(RiskParameters riskParameters, ) = getRiskParameters(0);` (Line 1634)

This means an attacker can **actively trigger** commission-burn distributions by initiating liquidation/forced-exercise flows when a target account is eligible, rather than passively waiting to sandwich a victim’s optional `builderCode==0` transaction.

🔗 Attack flow

Passive MEV Sandwich (around user transaction with `builderCode = 0`):

1. **Victim tx pending:** a user submits an option interaction via `PanopticPool.dispatch(...)` with `builderCode = 0`.
2. **Attacker front-runs:** attacker calls `CollateralTracker.deposit(...)` to mint a very large share balance right before the victim tx executes.
3. **Victim tx executes:** `PanopticPool` calls `CollateralTracker.settleMint/settleBurn` and the commission path burns `sharesToBurn` from the option owner (supply decreases, assets do not).
4. **Attacker back-runs:** attacker unwinds, capturing the share price uplift.

Active Trigger (Liquidation-style):

1. Attacker identifies a margin-called account eligible for liquidation.
2. Attacker deposits large liquidity into the relevant `CollateralTracker` (s) to dominate share supply.
3. Attacker calls `dispatchFrom`, which routes into `_liquidate(...)` and burns all positions via `_burnAllOptionsFrom(...)` using hardcoded `getRiskParameters(0)`.
4. `CollateralTracker.settleBurn(...)` charges commission; if `feeRecipient==0`, it burns shares.
5. Attacker exits when liquidity permits; the economic transfer happens at burn time.

How the attacker finds margin-called accounts (practically):

- Off-chain: call `PanopticPool.getAccumulatedFeesAndPositionsData(...)` + `RiskEngine.isAccountSolvent(...)` / `RiskEngine.getMargin(...)` to evaluate solvency at relevant ticks.
- On-chain: simply attempt `dispatchFrom(...)` for a candidate account; the call reverts unless the account is fully margin-called, so failed attempts are self-pruning.

The attacker's captured value is approximately:

$$\$ \$ \text{captured} \approx \text{donationValue} \times \frac{\text{attackerShares}}{\text{totalSharesAfterAttackerDeposit}} \$ \$$$

With sufficient capital, the attacker can make the ratio arbitrarily close to 1 and capture almost all of the distribution.

Capital requirements (Passive JIT capture):

For a pool with incumbent TVL `T` (in assets) and an attacker deposit `A`, the attacker's capture fraction at the instant of the burn is approximately:

$$\$ \$ \text{captureFraction} \approx \frac{A}{T + A} \$ \$$$

So to capture `p` of a burn event, the attacker needs roughly:

$$\$ \$ A \approx \frac{p}{1-p} T \$ \$$$

Examples: `p=90%` → `A≈9×T`; `p=99%` → `A≈99×T`. The PoC uses large deposits to make the capture fraction very close to 1 and show the asymptote clearly.

Flash loan severity multiplier:

Flash loans are **not** a universal multiplier for a passive mempool sandwich (the attacker must hold shares across a *separate* victim transaction). However, when the attacker can **actively trigger** the commission-burn event (e.g., via liquidation / force exercise paths that hardcode `getRiskParameters(0)`), they can wrap “enter → trigger burn → exit” in a single transaction, making **flash-loan-funded entry feasible**.

Practical constraints:

- Realized, in-tx profit is bounded by withdrawable liquidity: `maxWithdraw/maxRedeem` are capped by `s_depositedAssets` (not `totalAssets()`, which includes `s_assetsInAMM` and unrealized interest), and withdrawals are blocked when `numberOfLegs(owner) != 0`.
- In liquidation / force-exercise flows, the protocol delegates virtual shares to the distressed account; any “burn” that only consumes phantom shares is compensated on `revoke()` (i.e., it does not persist as a net `totalSupply()` decrease). The extractable “donation size” depends on how many *real* shares are actually burned.

- **Systematic MEV capture of commission distributions:** value intended for longer-term LPs can be captured by JIT entrants.
- **Fee self-rebating / fee bypass:** large or flash-funded actors can materially reduce effective commission paid, undermining the fee model.
- **LP incentive degradation:** Passive LP returns become MEV-farmable, reducing the attractiveness of providing liquidity.
- **MEV arms race:** Sophisticated searchers can bundle entry/exit around high-value option interactions.
- **MEV around risk events:** forced-exercise/liquidation flows can become predictable “donation” farming opportunities when they route through `getRiskParameters()`.

🔗 Recommended Mitigation Steps

There is no free lunch here: “burn-based instant distribution to current shareholders” is fundamentally JIT-capturable. Mitigations are about making the distribution not instantaneous or not purely share-price based.

1. Disallow burn-based distribution for commissions (Strongest, Simplest):

- Treat `builderCode == 0` as “protocol receives fees” rather than “burn to LPs”.
- Enforce `feeRecipient != 0` for all dispatch paths, or route `builderCode == 0` to a non-zero treasury recipient, then distribute via a separate time-weighted mechanism.

2. Stream/Vest distributions instead of instantaneous uplift:

- Accumulate commissions in a buffer and drip them into `totalAssets()` over a window (e.g., X blocks).
- This makes same-block JIT capture unprofitable because the “uplift” is smeared over time.

3. Cooldown / Time-weighting (Partial mitigation):

- A 1-block withdrawal cooldown reduces risk-free atomic sandwiches but does not remove the economic issue (attackers can hold across blocks).
- If implemented, it must also cover transfers (otherwise deposit→transfer→withdraw bypass exists).

🔗 Proof of Concept

Setup instructions:

1. Create the PoC file at `test/foundry/poc/CollateralTrackerCommissionBurnJITPoC.t.sol`
2. Apply the diff below (or copy the full file content)
3. Run the tests via `forge test --match-path test/foundry/poc/CollateralTrackerCommissionBurnJITPoC.t.sol -vvv`

Note: This PoC uses a minimal mock `PanopticPool` solely to satisfy `onlyPanopticPool` and to simulate the liquidation-style `delegate()` → `settleBurn()` → `revoke()` sequence that the real `PanopticPool` uses during `dispatchFrom` liquidations/forced exercises. A full `dispatchFrom` end-to-end liquidation PoC would require spinning up real option positions + solvency/oracle plumbing (typically via a forked environment) and is orthogonal to demonstrating the JIT-capturable share-burn mechanism inside `CollateralTracker`.

PoC diff:

[View detailed Proof of Concept](#)

Expected test output:

All 10 tests pass, demonstrating:

1. `test_PoC_CommissionBurn_JITCapture_WhenFeeRecipientIsZero` - JIT attacker captures >99% of commission burn distribution.
2. `test_PoC_SettleBurn_NoRealizedPremium_NoCommissionBurn` - Confirms `settleBurn` gating on `realizedPremium != 0`.
3. `test_PoC_ActiveCapture_SettleBurn_CommissionBurn_JITCapture` - Active attacker-initiated burn capture.
4. `test_PoC_FeeSelfRebating_PreDepositReducesEffectiveFee` - Fee payer can recapture >99% of their own fee.
5. `test_PoC_FeeSelfRebating_CostDecreases_WithShareFraction` - Effective fee decreases as the fee payer's share fraction increases.
6. `test_PoC_FlashLoan_ActiveCapture_IsAtomic_WhenLiquidityAllows` - Atomic flash loan attack variant.
7. `test_PoC_LiquidationStyle_DelegateRevoke_PhantomBurnIsRestored_WhenNoRealShares` - Phantom burn is restored on revoke (no persistent supply decrease).
8. `test_PoC_LiquidationStyle_DelegateRevoke_RealBurnPersists_WhenVictimHasRealShares` - Persistent supply decrease requires burning real shares.
9. `test_PoC_UpliftMayBeNonWithdrawable_WhenAssetsInAMMHigh` - Shows `maxWithdraw/maxRedeem` are capped by `s_depositedAssets` (not `totalAssets()`).
10. `test_PoC_RiskEngine_builderCode0_feeRecipient0` - Confirms `builderCode == 0` → `feeRecipient == address(0)`.

Panoptic marked as informative.

[M-16] Force exercise lacks caller-side bounds for exercise fee

Submitted by [Nyx](#)

`PanopticPool.sol` [L1598-L1664](#)

Summary

The force exercise path (`dispatchFrom()` → `_forceExercise()`) computes an `exerciseCost()` and immediately settles it without any caller-provided bounds on the maximum amount the caller is willing to pay. Because the fee depends on `currentTick`, `twapTick`, and per-leg liquidity deltas, the final amount the caller pays may change.

Finding description

The force exercise mechanism computes a fee (`exerciseFees`) that the force exercisor may pay to the position owner, and then redistributes/settles deltas via `getRefundAmounts()` and `refund()`. In `_forceExercise()`, the caller has no way to enforce a maximum fee or a maximum per-token outflow:

```
function _forceExercise(
    address account,
    TokenId tokenId,
    int24 twapTick,
    int24 currentTick
) internal {
    // ...
    exerciseFees = riskEngine().exerciseCost(
        currentTick,
        twapTick,
        tokenId,
        positionBalance
    );
    // ...
    LeftRightSigned refundAmounts = riskEngine().getRefundAmounts(
        account,
```

```

        exerciseFees,
        twapTick,
        ct0,
        ct1
    );
    ct0.refund(account, msg.sender, refundAmounts.rightSlot());
    ct1.refund(account, msg.sender, refundAmounts.leftSlot());
    // ...
}

```

The fee can change to both (a) whether any long leg is in-range at `currentTick`, and (b) the delta between the token amounts corresponding to a long leg's liquidity at `currentTick` versus `oracleTick` (here `twapTick`):

The force exercise fee is expected to vary with how far the position is from the money, which implies natural fee volatility with price movements $\propto$. Without a max-fee / min-receive bound, callers cannot protect themselves from changes.

A caller initiating a force exercise may end up paying more than anticipated. This can make position management more complex/unexpected for users who want to force exercise another position.

🔗 Recommended mitigation steps

Consider adding explicit caller-side bounds for force exercise payments.

🔗 Proof of Concept

In `PanopticPool.t.sol`, import a `PositionBalanceLibrary`:

```
"import {PositionBalance, PositionBalanceLibrary} from "@types/PositionBalance.sol";"
```

[View detailed Proof of Concept](#)

Panoptic marked as informative.

🔗 [M-17] High divergence check in `isSafeMode` is unreachable dead code providing false sense of security

Submitted by [anchabadze](#)

`RiskEngine.sol` [L929](#)

🔗 Finding description

The `highDivergence` check in `RiskEngine::isSafeMode()` is designed to detect when the median tick deviates significantly from the slow EMA by comparing `|medianTick - slowEMA| > 1906` ticks. This condition is intended to add +1 to `safeMode`, potentially pushing it to 3 (Lock Mode), which would cause minting operations to revert with `StaleOracle`.

However, this condition is mathematically and empirically proven to be unreachable dead code. Both `medianTick` and `slowEMA` are updated using the same `clampedTick` value, which is capped at `MAX_CLAMP_DELTA = 149` ticks per 64-second epoch. Due to this shared input and capping mechanism, their maximum possible divergence is mathematically bounded.

In equilibrium with constant price movement at maximum speed, `slowEMA` lags behind the current tick by approximately 1397 ticks, while `medianTick` (median of 8 observations) lags by approximately 522 ticks. This results in a maximum achievable divergence of approximately 875 ticks in equilibrium, or up to 935 ticks when accounting for transient effects. The threshold of 1906 ticks is unreachable, representing only 42-49% of what would be needed to trigger the condition.

Comprehensive testing across multiple extreme scenarios (flash loan attacks, post-initialization manipulation, price oscillation) confirmed that the divergence never exceeds 815 ticks, making the 1906-tick threshold impossible to reach under any realistic conditions.

In the provided PoC you can find two tests that prove this claim:

`test_MathematicalProof_MaximumDivergence()` which calculates the theoretical maximum divergence using EMA update formulas and equilibrium analysis, proving the threshold is unreachable by a factor of 2.04x, and

`test_HighDivergence_IsDeadCode()` which empirically verifies the threshold is unreachable by testing extreme scenarios and comparing results with theoretical predictions, confirming maximum observed divergence of ~815 ticks.

Scenario

1. The protocol assumes the `highDivergence` check provides protection against oracle staleness by comparing `medianTick` and `slowEMA` to detect significant divergence.
2. Both metrics are updated in `OraclePack::insertObservation()` using the same `clampedTick` value, which is limited by `clampTick()` to a maximum change of 149 ticks per epoch.
3. The `clampedTick` is calculated from the current tick but clamped to be within 149 ticks of the last observed tick, ensuring both `medianTick` and `slowEMA` always move in the same direction with bounded difference.
4. Even under extreme market conditions or deliberate manipulation attempts, the divergence between `medianTick` and `slowEMA` stabilizes at approximately 726-815 ticks, well below the 1906-tick threshold.
5. The `highDivergence` threshold of 1906 ticks ($2 \times \text{MAX_TICKS_DELTA}$) remains permanently unreachable, making this check dead code that never executes.
6. The protocol documentation and code comments suggest this check provides protection, creating a false sense of security while offering no actual protection.

Impact

The protocol documentation and code suggest that `highDivergence` provides protection against oracle manipulation, but this protection is completely non-functional.

The `safeMode` mechanism was designed with three additive conditions (`externalShock`, `internalDisagreement`, `highDivergence`). With `highDivergence` being dead code, `safeMode` can never reach 3 through oracle-based detection

Users may be able to mint positions during oracle manipulation scenarios that should be blocked, as the intended third layer of protection is non-functional.

Recommended mitigation steps

Recalculate the threshold based on actual achievable divergence. A threshold of approximately 600-700 ticks would be reachable under extreme conditions while still providing meaningful protection or use different metrics

Proof of Concept

1. Create a test file `HighDivergenceTest.t.sol` in `test/foundry/core/`.
2. Paste the PoC code provided below into the file.
3. Save the file.
4. Run the test: `ETH_RPC_URL="<YOUR_MAINNET_RPC_URL>" forge test --match-path test/foundry/core/HighDivergenceTest.t.sol -vv`.

[View detailed Proof of Concept](#)

Panoptic marked as informative.

[M-18] Solvency tick divergence blind spot in `RiskEngine.getSolvencyTicks`

Submitted by [qed](#)

`RiskEngine.sol` [L962-L967](#)

The `getSolvencyTicks()` function uses a 3D squared-Euclidean-norm gate centered on the median tick to decide whether to check solvency at multiple ticks. However, symmetric tick configurations can bypass this

gate even when `|spotTick - currentTick|` significantly exceeds `MAX_TICKS_DELTA`, potentially allowing positions to pass solvency checks at a single tick when they would be insolvent at the current tick.

🔗 Finding description

1. The squared-distance-from-median gate can be bypassed with symmetric tick arrangements:

```
// contracts/RiskEngine.sol:962-978
function getSolvencyTicks(
    int24 currentTick,
    OraclePack _oraclePack
) external view returns (int24[] memory, OraclePack) {
    (int24 spotTick, int24 medianTick, int24 latestTick, OraclePack oraclePack) = _oraclePack
        .getOracleTicks(currentTick, EMA_PERIODS, MAX_CLAMP_DELTA);

    int24[] memory atTicks;

    // Gate uses sum of squared distances from median
    if (
        int256(spotTick - medianTick) ** 2 +
        int256(latestTick - medianTick) ** 2 +
        int256(currentTick - medianTick) ** 2 >
        MAX_TICKS_DELTA ** 2
    ) {
        // Multi-tick check (4 ticks)
        atTicks = new int24[](4);
        // ...
    } else {
        // Single tick check (only spotTick)
        atTicks = new int24[](1);
        atTicks[0] = spotTick;
    }
    // ...
}
```

2. Mathematical blind spot with symmetric configurations:

If `medianTick = M`, `latestTick = M`, and:

- `spotTick = M + 0.70 * MAX_TICKS_DELTA`
- `currentTick = M - 0.70 * MAX_TICKS_DELTA`

Then the squared norm calculation:

$$(0.70 * D)^2 + 0^2 + (-0.70 * D)^2 = 0.98 * D^2 < D^2$$

The gate does NOT fire, but `|spotTick - currentTick| = 1.40 * D > D` is a significant divergence.

3. Discrepancy with `isSafeMode` logic:

Notably, `isSafeMode()` DOES include a check for `|currentTick - spotTick|` divergence:

```
// contracts/RiskEngine.sol:903
if (Math.abs(currentTick - spotTick) > MAX_TICKS_DELTA) {
    // External shock detected - safe mode ON
}
```

But `getSolvencyTicks()` does not use this same check. This creates an inconsistency where the protocol may recognize an external shock (safe mode ON) but still only check solvency at a single tick.

🔗 Attack scenario

Step 1: Market conditions create a symmetric tick divergence around the median:

- `medianTick = 0` (from TWAP)
- `latestTick = 0` (same as median)
- `spotTick = +667` ($0.7 * 953$)

- `currentTick = -667`

Step 2: The squared norm calculation:

$$(667)^2 + 0 + (-667)^2 = 889,778 < 908,209 = 953^2$$

The gate doesn't trigger - only `spotTick` (667) is returned for solvency checks.

Step 3: A position that is:

- Solvent at `spotTick = 667`
- Insolvent at `currentTick = -667`

...passes the solvency check because only `spotTick` is evaluated.

Step 4: The gap between spot and current tick is 1,334 ticks (`667 - (-667)`), which is 140% of `MAX_TICKS_DELTA` (953). This represents a significant price deviation that should trigger multi-tick evaluation.

Impact

1. **Solvency check bypass:** Positions may pass solvency checks when evaluated only at `spotTick` while being insolvent at `currentTick`.
2. **Inconsistent risk assessment:** The protocol uses different divergence logic in `isSafeMode()` vs `getSolvencyTicks()`, creating a gap where safe mode is active but solvency is only checked at one tick.
3. **Directional portfolio vulnerability:** Portfolios with directional exposure (heavily long or short) are particularly sensitive to tick-specific solvency. Checking only at `spotTick` can miss insolvency at the actual current price.

Recommended mitigation steps

Add an explicit pairwise check for spot-current divergence to align with `isSafeMode` logic:

```
function getSolvencyTicks(
    int24 currentTick,
    OraclePack _oraclePack
) external view returns (int24[] memory, OraclePack) {
    (int24 spotTick, int24 medianTick, int24 latestTick, OraclePack oraclePack) = _oraclePack
        .getOracleTicks(currentTick, EMA_PERIODS, MAX_CLAMP_DELTA);

    int24[] memory atTicks;

    int256 d1 = int256(spotTick - medianTick);
    int256 d2 = int256(latestTick - medianTick);
    int256 d3 = int256(currentTick - medianTick);
    int256 d4 = int256(currentTick - spotTick); // Pairwise check

    if (
        d1 ** 2 + d2 ** 2 + d3 ** 2 > MAX_TICKS_DELTA ** 2 ||
        Math.abs(d4) > MAX_TICKS_DELTA // Align with isSafeMode logic
    ) {
        // Multi-tick check
        atTicks = new int24[](4);
        // ...
    } else {
        atTicks = new int24[](1);
        atTicks[0] = spotTick;
    }
    // ...
}
```

This ensures that any significant divergence between spot and current tick triggers multi-tick solvency evaluation, consistent with how `isSafeMode` detects external shocks.

Summary

`RiskEngine.getSolvencyTicks()` uses a 3D squared-Euclidean-norm gate that can be bypassed with symmetric tick configurations where spot and current ticks are on opposite sides of the median. This allows positions to be evaluated at only the spot tick even when `|spotTick - currentTick|` significantly exceeds `MAXTICKSDELTA` (953 ticks). The fix is to add an explicit pairwise spot-current check, aligning with the logic already present in `isSafeMode()`.

Proof of Concept

File: `test/foundry/core/PoC_SolvencyBlindSpot.t.sol`

Run with: `forge test --match-test test_poc_sololvencyBlindSpot -vvv`

[View detailed Proof of Concept](#)

[Panoptic mitigated:](#)

Check solvency at 4 ticks when `safeMode > 0`.

Status: Mitigation confirmed. Full details in reports from [Valves](#), [edoscoba](#), and [Nyx](#).

[M-19] Withdrawing just before a bad debt event can increase losses for remaining liquidity providers

Submitted by [Valves](#)

`CollateralTracker.sol` [L1262](#)

Summary

When large positions are heading toward bad debt, liquidity providers can withdraw just before loss socialization. This timing advantage lets exiting LPs avoid the loss, concentrating it on the remaining LPs and resulting in unfair loss distribution by front-running the bad debt event.

Impact

A malicious LP never incurs losses from bad debt socialization, while the remaining LPs bear the cost instead.

Finding description

In Panoptic's liquidation flows, pool deficits are socialized when the liquidatee does not have enough funds (shares and assets) to cover the liquidator bonus. If an LP exits immediately before socialization, their shares are not impacted by the imminent deficit, shifting the entire loss burden onto LPs who remain.

Original code block has been omitted. [View complete submission](#).

When a position becomes liquidatable and has accumulated bad debt and the liquidatee does not have enough funds to transfer to the liquidator, new shares are minted to the liquidator to cover the bonus for the liquidation. This way we see an increase of the `totalShares` meanwhile the `totalAssets` go down because it is also transferred out of the protocol to the liquidator. The ratio between shares and assets change which can be front-run by LPs to avoid the bad debt allocation event.

Recommended mitigation steps

Implement a mechanism to fairly distribute losses from bad debt among all liquidity providers, possibly by locking withdrawals for a short period or pro-rating losses based on withdrawal timing.

Proof of Concept

Add the helper and the test inside `CollateralTracker.t.sol`:

[View detailed Proof of Concept](#)

Output:

The withdrawal amount available to Charlie immediately before liquidation is greater than the amount available immediately after liquidation, demonstrating that exiting LPs avoid the deficit while remaining LPs bear its full impact.

```
totalAssets before the liquidation: 11000000000000000001
totalShares before the liquidation: 1089910000000000000100000
Charlie can front run the bad debt and can withdraw: 1009257645126661834463
Charlie missed the front run opportunity and now can withdraw 989327316140468395517
totalAssets after the liquidation: 100000000000000000002
totalShares after the liquidation: 1010787818839539913079152234
```

Panoptic disputed

🔗 Low Risk and Informational Issues

For this audit, 108 QA reports were submitted by wardens compiling low risk and informational issues. The [QA report highlighted below](#) by jerry0422 received the top score from the judge. 41 Low-severity findings were also submitted individually, and can be viewed [here](#).

The following wardens also submitted QA reports: [OxOburn](#), [Oxanony](#), [Oxcode](#), [OxFBI](#), [Oxhp9](#), [Oxki](#), [Oxnija](#), [Oxvictorsr](#), [adeolu](#), [adexgee](#), [Afriauditor](#), [AgengDev](#), [Agontuk](#), [Agrawain](#), [Ahmerdrarerh](#), [aliabouzeid](#), [allan31](#), [anon1one](#), [AriF9212](#), [Arrow](#), [aster](#), [Auttrs](#), [blackgrease](#), [Brainiac001](#), [Budaishere](#), [BugNet](#), [caesar49](#), [Charming](#), [chuvak](#), [cosin3](#), [cybertechajju](#), [DevBear0411](#), [Diavolo](#), [Dps4356](#), [edoscoba](#), [eightzerofour](#), [Engama](#), [erpal](#), [EVDoc](#), [Fatma](#), [felconsec](#), [francoHacker](#), [freebird0323](#), [golu25012000](#), [Henri](#), [hiram](#), [hirusha](#), [home1344](#), [I1iveF0rTh1Sh1t](#), [johnyfwesh](#), [justingoro](#), [K42](#), [KineticsOfWeb3](#), [kishorsinghpatel](#), [kl4r10n](#), [legat](#), [Maheskarre](#), [Make1Chop](#), [mayursinh](#), [mccarthyquiloX](#), [mddragon18](#), [Meks079](#), [merlin_san](#), [mijaluz](#), [MinionTechs](#), [Mnemor](#), [MrFlickery](#), [Nepker](#), [NI97](#), [niffylord](#), [Nkolv](#), [Oxseenerh](#), [padma](#), [PillarsOfLight](#), [prk0](#), [ProngsDev](#), [qed](#), [Race](#), [raz-uh](#), [ref2basic](#), [reverb006](#), [Rifter](#), [SAGEisbuilding](#), [sam](#), [saneryee](#), [SarveshLimaye](#), [sexretxt](#), [shanemi](#), [slvDev](#), [SnowX](#), [spectator](#), [Specter07](#), [spikeDu](#), [TOSHI](#), [Trynax](#), [udaykiranpedda](#), [UnvirsalX](#), [v12](#), [valarislife](#), [Vemus](#), [Venkat5599](#), [Vinay](#), [WinningTeam](#), [Wizax](#), [xmaryo](#), [Yu4n](#), and [Zhenyazhd](#).

- 🔗 [01] `validateCollateralWithdrawable()` hardcodes `safeMode=0`, bypassing stricter collateral requirements during volatile conditions

PanopticPool.sol [L420](#)

🔗 Finding description and impact

The `validateCollateralWithdrawable()` function hardcodes `safeMode = 0` when calling `_validateSolvency()`:

This is inconsistent with `dispatch()`, which correctly passes the current safe mode level:

When `safeMode > 0`, the `_checkSolvencyAtTicks()` function enforces stricter collateral requirements by setting utilization to 100%, disabling cross-margining benefits:

```
if (safeMode > 0) {
    unchecked {
        uint32 maxUtilizations = uint32(DECIMALS + (DECIMALS << 16));
        positionBalanceArray[0] = PositionBalanceLibrary.storeBalanceData(
            positionBalanceArray[0].positionSize(),
            maxUtilizations,
            0
        );
    }
}
```

By hardcoding `safeMode = 0`, withdrawals always use relaxed collateral requirements even when the protocol is in safe mode (triggered by oracle volatility or guardian lock). This allows users to withdraw more collateral than intended during high volatility periods, potentially leaving the protocol in a worse state. The `validateCollateralWithdrawable()` function is called by `CollateralTracker.withdraw()` and `CollateralTracker.redeem()` when users have open positions, making this a practical bypass of safe mode protections.

Recommended mitigation steps

Pass the actual `safeMode` value from `riskParameters` instead of hardcoding 0:

```
function validateCollateralWithdrawable(
    address user,
    TokenId[] calldata positionIdList,
    bool usePremiaAsCollateral
) external view {
    (RiskParameters riskParameters, ) = getRiskParameters(0);
    _validateSolvency(
        user,
        positionIdList,
        riskParameters.bpDecreaseBuffer(),
        usePremiaAsCollateral,
        - 0
        + riskParameters.safeMode()
    );
}
```

[02] Missing zero shares check in `deposit()` allows users to lose deposited assets

`CollateralTracker.sol` [L562-L577](#)

Finding description

The `deposit()` function in `CollateralTracker.sol` validates that assets `!= 0` but does not validate that the resulting shares from `previewDeposit()` is non-zero:

```
function deposit(uint256 assets, address receiver) external payable returns (uint256 shares) {
    _accrueInterest(msg.sender, IS_DEPOSIT);
    if (assets > type(uint104).max) revert Errors.DepositTooLarge();
    if (assets == 0) revert Errors.BelowMinimumRedemption();

    shares = previewDeposit(assets); // @audit can return 0

    // ... assets transferred ...

    _mint(receiver, shares); // @audit mints 0 shares
    s_depositedAssets += uint128(assets);
    // ...
}
```

The `previewDeposit()` function uses `Math.mulDiv` which rounds down:

```
function previewDeposit(uint256 assets) public view returns (uint256 shares) {
```

```

    shares = Math.mulDiv(assets, totalSupply(), totalAssets());
}

```

When `assets * totalSupply() < totalAssets()`, the result rounds down to zero. This can occur when:

- The share price has appreciated significantly (e.g., through interest accrual or donations)
- A user deposits a relatively small amount of assets

In this scenario, the user's assets are transferred to the pool and `s_depositedAssets` is incremented, but the user receives zero shares in return, resulting in complete loss of the deposited funds. While the protocol initializes with virtual shares (`106`) and virtual assets (1) to mitigate inflation attacks, over time as `totalAssets` grows through interest accrual (`unrealizedInterest`) or donations, small deposits can still round to zero shares.

Impact

Users can lose their entire deposit if the amount is too small relative to the current share price.

Recommended mitigation steps

Add a check to revert when zero shares would be minted:

```

function deposit(uint256 assets, address receiver) external payable returns (uint256 shares) {
    _accrueInterest(msg.sender, IS_DEPOSIT);
    if (assets > type(uint104).max) revert Errors.DepositTooLarge();
    if (assets == 0) revert Errors.BelowMinimumRedemption();

    shares = previewDeposit(assets);
    if (shares == 0) revert Errors.BelowMinimumRedemption(); // @audit add this check

    // ... rest of function
}

```

[03] `CommissionPaid` event emits `protocolSplit` instead of `builderSplit` for `commissionPaidBuilder` parameter

- `CollateralTracker.sol` [L1577](#)
- `CollateralTracker.sol` [L1656](#)

Finding description and impact

The `CommissionPaid` event is defined with four parameters, where `commissionPaidBuilder` should represent the amount of assets paid to the builder:

```

/// @param commissionPaidProtocol The amount of assets paid that goes to the PLPs (if builder == address(0))
/// @param commissionPaidBuilder The amount of assets paid that goes to the builder
event CommissionPaid(
    address indexed owner,
    address indexed builder,
    uint128 commissionPaidProtocol,
    uint128 commissionPaidBuilder
);

```

However, when emitting the event in `settleMint()` and `settleBurn()`, the code incorrectly uses `protocolSplit()` for both parameters instead of using `builderSplit()` for the fourth parameter:

```

emit CommissionPaid(
    optionOwner,
    address(uint160(riskParameters.feeRecipient())),
    uint128((commissionFee * riskParameters.protocolSplit()) / DECIMALS),
    uint128((commissionFee * riskParameters.protocolSplit()) / DECIMALS) // @audit should be builderSplit
);

```

Note that the actual token transfers are performed correctly—the protocol receives `protocolSplit()` and the builder receives `builderSplit()`. Only the emitted event data is incorrect.

This causes off-chain systems, indexers, and front-ends that rely on these events to display incorrect commission distribution data. Users and builders monitoring their fee earnings will see inaccurate values.

🔗 Recommended mitigation steps

Replace `protocolSplit()` with `builderSplit()` for the `commissionPaidBuilder` parameter in both locations:

```
emit CommissionPaid(
    optionOwner,
    address(uint160(riskParameters.feeRecipient())),
    uint128((commissionFee * riskParameters.protocolSplit()) / DECIMALS),
-   uint128((commissionFee * riskParameters.protocolSplit()) / DECIMALS)
+   uint128((commissionFee * riskParameters.builderSplit()) / DECIMALS)
);
```

🔗 [04] `unlockPool()` emits `GuardianSafeModeUpdated(true)` instead of `false` when lifting the lock

`RiskEngine.sol` [L243](#)

🔗 Finding description and impact

The `GuardianSafeModeUpdated` event is defined with a `lockMode` parameter that indicates the state of the safe mode lock:

```
/// @param lockMode True when safe mode is forcibly locked, false when the lock is lifted.
event GuardianSafeModeUpdated(bool lockMode);
```

However, `unlockPool()` emits true instead of false:

```
function lockPool(PanopticPool pool) external onlyGuardian {
    emit GuardianSafeModeUpdated(true); // Correct: locking emits true
    pool.lockSafeMode();
}

function unlockPool(PanopticPool pool) external onlyGuardian {
    emit GuardianSafeModeUpdated(true); // Incorrect: unlocking should emit false
    pool.unlockSafeMode();
}
```

Per the event's NatSpec documentation:

- true = “safe mode is forcibly locked”
- false = “lock is lifted”

Since `unlockPool()` lifts the lock, it should emit false, not true. This appears to be a copy-paste error from `lockPool()`. Off-chain systems, monitoring tools, and front-ends relying on this event to track the safe mode state of pools will receive incorrect information, showing pools as locked when they have actually been unlocked.

🔗 Recommended mitigation steps

```
function unlockPool(PanopticPool pool) external onlyGuardian {
-   emit GuardianSafeModeUpdated(true);
+   emit GuardianSafeModeUpdated(false);
    pool.unlockSafeMode();
}
```

🔗 [05] Memory copy of `premierByLeg` inside loop in `haircutPremia()` causes redundant array copying on each iteration

Finding description

In `RiskEngine.haircutPremia()`, a memory copy of the `premierByLeg` array is created inside the outer loop at line 740:

```
for (uint256 i = 0; i < positionIdList.length; i++) {
    tokenId = positionIdList[i];
    LeftRightSigned[4][] memory _premierByLeg = premierByLeg; // @audit: copied on every iteration
    for (uint256 leg = 0; leg < tokenId.countLegs(); ++leg) {
        if (
            tokenId.isLong(leg) == 1 &&
            LeftRightSigned.unwrap(_premierByLeg[i][leg]) != 0
        ) {
            // ... uses _premierByLeg[i][leg]
        }
    }
}
```

Since `_premierByLeg` is declared inside the outer for loop, a new memory copy of the entire `premierByLeg` array is created on every iteration. This is unnecessary because:

1. The `premierByLeg` parameter is already in memory (passed as memory)
2. The copy `_premierByLeg` is only read, never modified
3. The same data is being copied repeatedly

This results in wasted gas proportional to `positionIdList.length * premierByLeg.length`, which can be significant during liquidations involving multiple positions.

Impact

Increased gas costs for liquidation operations. While this does not put assets at direct risk, it increases the cost of liquidations which are time-sensitive operations. In extreme cases with many positions, this could make liquidations more expensive than necessary.

Recommended mitigation steps

Move the memory copy outside the outer loop so it is only created once:

```
LeftRightSigned[4][] memory _premierByLeg = premierByLeg; // Move outside loop
for (uint256 i = 0; i < positionIdList.length; i++) {
    tokenId = positionIdList[i];
    for (uint256 leg = 0; leg < tokenId.countLegs(); ++leg) {
        if (
            tokenId.isLong(leg) == 1 &&
            LeftRightSigned.unwrap(_premierByLeg[i][leg]) != 0
        ) {
            // ...
        }
    }
}
```

Alternatively, since `premierByLeg` is already a memory parameter and is only being read, the local copy can be removed entirely and `premierByLeg` can be used directly.

[06] `PanopticPool.onERC1155Received` accepts tokens from any ERC1155 contract without sender validation

Finding description and impact

The `onERC1155Received` function in `PanopticPool.sol` unconditionally returns the success selector without validating that `msg.sender` is the canonical SFPM contract:

```
function onERC1155Received(
    address,
    address,
    uint256,
    uint256,
    bytes memory
) external pure returns (bytes4) {
    return this.onERC1155Received.selector;
}
```

Per the ERC1155 standard, when a contract receives tokens via `safeTransferFrom` or `_mint`, the receiving contract's `onERC1155Received` hook is called with `msg.sender` being the ERC1155 contract initiating the transfer. The function should validate this sender to ensure tokens are only accepted from expected sources.

The current implementation allows any ERC1155 contract to transfer tokens to the `PanopticPool`, which will be permanently stuck since there is no mechanism to recover arbitrary ERC1155 tokens. While the protocol's core accounting relies on internal mappings (`s_positionBalance`, `s_positionsHash`) rather than ERC1155 token balances, this represents a deviation from secure receiver patterns and violates the principle of defense in depth.

The NatSpec comment explicitly states the function "Returns magic value when called by the SemiFungiblePositionManager contract" but does not enforce this constraint.

🔗 Recommended mitigation steps

Add sender validation to ensure only the SFPM can trigger successful token receipts:

```
function onERC1155Received(
    address,
    address,
    uint256,
    uint256,
    bytes memory
) external view returns (bytes4) {
    if (msg.sender != address(SFPM)) revert Errors.NotAuthorized();
    return this.onERC1155Received.selector;
}
```

🔗 [07] `deposit()` and `mint()` are payable but do not refund ETH for ERC20 collateral, causing user funds to be trapped

- `CollateralTracker.sol` [L557](#)
- `CollateralTracker.sol` [L611](#)
- `CollateralTracker.sol` [L465-L474](#)

🔗 Finding description

The `deposit()` and `mint()` functions in `CollateralTracker.sol` are marked as payable to support native ETH collateral in Uniswap V4 pools. However, when the underlying collateral is an ERC20 token, any ETH sent with these calls is silently accepted and permanently trapped in the contract. For the V4 path, the `unlockCallback()` function only refunds surplus ETH when the underlying asset is native ETH (`isAddressZero()` returns `true`):

```
if (Currency.wrap(underlyingAsset).isAddressZero()) {
    poolManager().settle{value: uint256(delta)}();
    uint256 surplus = valueOrigin - uint256(delta);
    if (surplus > 0) SafeTransferLib.safeTransferETH(account, surplus);
} else {
    // ERC20 path - valueOrigin (msg.value) is completely ignored
    poolManager().sync(Currency.wrap(underlyingAsset));
    SafeTransferLib.safeTransferFrom(underlyingAsset, account, address(poolManager()), uint256(delta));
    poolManager().settle();
}
```

For the V3 path (`poolManager == address(0)`), any ETH sent remains in the `CollateralTracker` with no mechanism to retrieve it. Since the `CollateralTracker` has no `receive()` function, withdrawal mechanism for trapped ETH, or validation to reject `msg.value > 0` for ERC20 collateral, any accidentally sent ETH is permanently lost.

Impact

Users who mistakenly send ETH when depositing ERC20 collateral will lose those funds. While this requires user error, the payable modifier creates an implicit expectation that ETH handling is supported, and the silent acceptance of ETH (rather than reverting) exacerbates the issue.

Recommended mitigation steps

Add validation at the start of `deposit()` and `mint()` to reject ETH when the collateral is not native ETH:

```
function deposit(uint256 assets, address receiver) external payable returns (uint256 shares) {
    if (msg.value > 0 && !Currency.wrap(underlyingToken()).isAddressZero()) {
        revert Errors.UnexpectedETH();
    }
    // ... rest of function
}
```

Alternatively, refund any `msg.value` in the ERC20 branch of `unlockCallback()`:

```
} else {
    poolManager().sync(Currency.wrap(underlyingAsset));
    SafeTransferLib.safeTransferFrom(underlyingAsset, account, address(poolManager()), uint256(delta));
    poolManager().settle();

    // Refund any accidentally sent ETH
    if (valueOrigin > 0) SafeTransferLib.safeTransferETH(account, valueOrigin);
}
```

[08] Median tick calculation rounds toward zero instead of floor for negative values

`OraclePack.sol` [L416](#)

Finding description and impact

In `OraclePack.sol`, the `getMedianTick` function calculates the median of 8 price observations by averaging the 3rd and 4th ranked residuals:

```
function getMedianTick(OraclePack oraclePack) internal pure returns (int24) {
    unchecked {
        int24 rank3 = oraclePack.residualTickOrdered(3);
        int24 rank4 = oraclePack.residualTickOrdered(4);

        int24 _referenceTick = oraclePack.referenceTick();

        return _referenceTick + ((rank3) + (rank4)) / 2;
    }
}
```

Solidity's signed integer division rounds toward zero rather than flooring. This creates an asymmetric rounding behavior:

- Positive odd sums round down: $3 / 2 = 1$
- Negative odd sums round up: $-3 / 2 = -1$ (floor would give -2)

For example, when `rank3 = -1` and `rank4 = 0`:

- Current behavior: $(-1 + 0) / 2 = 0$
- Floor behavior: $(-1 + 0) / 2 = -1$

This results in a systematic bias toward zero of up to 1 tick when the sum of the two middle residuals is an odd negative number. While the magnitude is small ($\sim 0.01\%$ price difference per tick), oracle calculations typically use consistent floor rounding to avoid any directional bias.

Recommended mitigation steps

Use arithmetic right shift which performs floor division for signed integers:

```
return _referenceTick + ((rank3 + rank4) >> 1);
```

Alternatively, implement explicit floor division:

```
int24 sum = rank3 + rank4;
return _referenceTick + (sum >= 0 ? sum / 2 : (sum - 1) / 2);
```

[09] `TokenId` validation allows `riskPartner` to reference inactive leg when index is 0

`TokenId.sol` [L507-L512](#)

Finding description

The `validate()` function in `TokenIdLibrary` checks that risk partners are mutual but does not verify that the `riskPartner` index points to an active leg:

```
uint256 riskPartnerIndex = self.riskPartner(i);
if (riskPartnerIndex != i) {
    // Ensures that risk partners are mutual
    if (self.riskPartner(riskPartnerIndex) != i)
        revert Errors.InvalidTokenIdParameter(3);
}
```

An inactive leg has all bits set to zero, meaning `riskPartner()` returns 0 for any inactive leg. This creates an edge case where leg 0 can set `riskPartner=2` (or any index pointing to an inactive leg), and the mutuality check passes because `riskPartner(inactiveLeg) == 0 == i` when `i=0`.

For example, with a 2-leg position (legs 0 and 1 active, legs 2 and 3 inactive):

- Leg 0: `riskPartner = 2` (pointing to inactive leg)
- Leg 2: inactive (all zeros, so `riskPartner = 0`)
- Mutuality check: `riskPartner(2) != 0 → 0 != 0 → false → no revert`

This allows a structurally inconsistent `TokenId` to pass validation where leg 0 believes it has a risk partner relationship with a non-existent leg.

Impact

The practical impact is limited because downstream usage in `RiskEngine._getRequiredCollateralSingleLegPartner()` performs additional checks that would fail for an inactive partner:

```
if (
    tokenId.asset(partnerIndex) == tokenId.asset(index) &&
    tokenId.optionRatio(partnerIndex) == tokenId.optionRatio(index)
)
```

Since an inactive leg has `optionRatio = 0` and an active leg requires `optionRatio > 0`, this condition fails and no improper margin reduction is applied. The malformed `TokenId` results in no collateral benefit to the user, making this a validation gap rather than an exploitable vulnerability.

🔗 Recommended mitigation steps

Add an explicit check that `riskPartnerIndex` points to an active leg:

```
uint256 riskPartnerIndex = self.riskPartner(i);
if (riskPartnerIndex != i) {
    // Ensure risk partner points to an active leg
    if (riskPartnerIndex >= self.countLegs())
        revert Errors.InvalidTokenIdParameter(3);
    // Ensures that risk partners are mutual
    if (self.riskPartner(riskPartnerIndex) != i)
        revert Errors.InvalidTokenIdParameter(3);
}
```

🔗 [10] Guardian `lockMode` not enforced in `getSolvencyTicks` allows weaker solvency checks during emergency lock

RiskEngine.sol [L947-L981](#)

🔗 Finding description and impact

The protocol has two independent safety mechanisms that should share a single source of truth but don't:

1. `isSafeMode()` (L908-940): Computes a `safeMode` level (`0-3+`) by checking oracle conditions AND the guardian's `lockMode`. When `lockMode = 3`, it adds 3 to `safeMode`, blocking new mints.
2. `getSolvencyTicks()` (L947-981): Determines whether to check solvency at 1 tick (normal) or 4 ticks (conservative) based solely on a 3D norm calculation of tick deviations. It completely ignores `lockMode`.

```
// isSafeMode() - correctly checks lockMode
932 uint8 lockMode = oraclePack.lockMode();
938 safeMode = ... + lockMode;
939
940 // getSolvencyTicks() - does NOT check lockMode
941 if (
942     int256(spotTick - medianTick) ** 2 +
943     int256(latestTick - medianTick) ** 2 +
944     int256(currentTick - medianTick) ** 2 >
945     MAX_TICKS_DELTA ** 2
946 ) {
947     // 4 ticks
948 } else {
949     // 1 tick - even when guardian has locked the pool
950 }
```

When the guardian calls `lockPool()`, the intention is to put the protocol into maximum protection mode. However:

- New mints: Blocked (correct, via `safeMode > 2`)
- Solvency checks: May still use only 1 tick (incorrect)

This inconsistency means that during an emergency lock—when the guardian has identified a threat requiring intervention—existing positions are still evaluated using potentially weaker solvency criteria. If the guardian locked preemptively before oracle metrics triggered the 3D norm threshold, or locked for reasons unrelated to tick divergence, solvency checks remain in “normal” mode, despite the emergency state. This could allow:

- Positions that should be liquidated to avoid liquidation.
- Unfair liquidations at manipulated tick values.
- General inconsistency between the protocol's stated safety posture and actual enforcement.

🔗 Recommended mitigation steps

Modify `getSolvencyTicks()` to check `lockMode` and force 4-tick solvency checks when the guardian has locked the pool:

```
function getSolvencyTicks(
    int24 currentTick,
    OraclePack _oraclePack
) external view returns (int24[] memory, OraclePack) {
    (int24 spotTick, int24 medianTick, int24 latestTick, OraclePack oraclePack) = _oraclePack
        .getOracleTicks(currentTick, EMA_PERIODS, MAX_CLAMP_DELTA);

    int24[] memory atTicks;

    if (
        _oraclePack.lockMode() == 3 || // @audit: Add lockMode check
        int256(spotTick - medianTick) ** 2 +
            int256(latestTick - medianTick) ** 2 +
            int256(currentTick - medianTick) ** 2 >
        MAX_TICKS_DELTA ** 2
    ) {
        atTicks = new int24[](4);
        atTicks[0] = spotTick;
        atTicks[1] = medianTick;
        atTicks[2] = latestTick;
        atTicks[3] = currentTick;
    } else {
        atTicks = new int24[](1);
        atTicks[0] = spotTick;
    }

    return (atTicks, oraclePack);
}
```

🔗 [11] `TARGET_UTILIZATION` NatSpec comment incorrectly states 90% instead of 66.67%

`RiskEngine.sol` [L174](#)

🔗 Finding description and impact

The `TARGET_UTILIZATION` constant has a NatSpec comment stating the target utilization is 90%, but the actual intended value is ~66.67%:

```
/// @notice Target utilization (scaled by WAD).
/// @dev Target utilization = 90%.
int256 public constant TARGET_UTILIZATION = 2 ether / int256(3);
```

The value `2 ether / int256(3)` equals `~0.667e18`, which represents 66.67% in WAD terms, not 90% as documented.

🔗 Recommended mitigation steps

Update the comment to reflect the actual value:

```
/// @notice Target utilization (scaled by WAD).
-/// @dev Target utilization = 90%.
+/// @dev Target utilization = 66.67%.
int256 public constant TARGET_UTILIZATION = 2 ether / int256(3);
```

🔗 [12] NatSpec comments in `Math.sol` incorrectly document boundary conditions for `getAmountsForLiquidity`

- `Math.sol` [L300](#)
- `Math.sol` [L322](#)
- `Math.sol` [L352](#)

🔗 Finding description and impact

The NatSpec comments for token amount calculation functions in `Math.sol` incorrectly document the boundary conditions, contradicting the actual implementation.

Line 300 (`getAmount0ForLiquidityUp`):

```
/// @return The amount of token0 represented by `liquidityChunk` when `currentTick < tickLower`
```

Line 336 (`getAmount0ForLiquidity`):

```
/// @return The amount of token0 represented by `liquidityChunk` when `currentTick < tickLower`
```

Line 322 (`getAmount1ForLiquidityUp`):

```
/// @return The amount of token1 represented by `liquidityChunk` when `currentTick > tickUpper`
```

Line 352 (`getAmount1ForLiquidity`):

```
/// @return The amount of token1 represented by `liquidityChunk` when `currentTick > tickUpper`
```

However, the actual implementation in `getAmountsForLiquidity` (lines 371-378) uses `<=` and `>=`:

```
if (currentTick <= liquidityChunk.tickLower()) {
    amount0 = getAmount0ForLiquidity(liquidityChunk);
} else if (currentTick >= liquidityChunk.tickUpper()) {
    amount1 = getAmount1ForLiquidity(liquidityChunk);
}
```

Boundary	NatSpec	Implementation
Lower	<	<=
Upper	>	>=

The implementation correctly matches Uniswap V3's `LiquidityAmounts.sol`, but the documentation is misleading.

🔗 Recommended mitigation steps

Update the NatSpec comments to accurately reflect the boundary conditions:

```
- /// @return The amount of token0 represented by `liquidityChunk` when `currentTick < tickLower`
+ /// @return The amount of token0 represented by `liquidityChunk` when `currentTick <= tickLower`
```

```
- /// @return The amount of token1 represented by `liquidityChunk` when `currentTick > tickUpper`
+ /// @return The amount of token1 represented by `liquidityChunk` when `currentTick >= tickUpper`
```

🔗 Mitigation Review

🔗 Introduction

Following the C4 audit, 4 wardens (Team [Valves](#) ([vesko210](#) and [Merulez99](#)), [edoscoba](#) and [Nyx](#)) reviewed the mitigations for all identified issues.

Additional details can be found within the Panoptic: Next Core Mitigation Review repositories:

- [Round 1](#)
- [Round 2](#)

🔗 Mitigation Review Scope & Summary

During the mitigation review, the wardens confirmed that all in-scope findings were mitigated. They also surfaced 1 new Low severity issue.

The table below provides details regarding the status of each in-scope vulnerability from the original audit, followed by full details on the new issues.

Original Issue	Status	Mitigation URL
H-01	🟢 Mitigation Confirmed	Commit 249fb90
H-02	🟢 Mitigation Confirmed	Commit 8d603d3
H-03	🟢 Mitigation Confirmed	Commit 5bf34b
M-01	🟢 Mitigation Confirmed	Commit a5cfc6
M-02	🟢 Mitigation Confirmed	Commit 4ef0998
M-03	🟢 Mitigation Confirmed	Commit bebe915
M-04	🟢 Mitigation Confirmed	Commit d434388
M-05	🟢 Mitigation Confirmed	Commit 6b154d9
M-06	🟢 Mitigation Confirmed	Commit 069c00b
M-07	🟢 Mitigation Confirmed	Commit fb73717
M-09	🟢 Mitigation Confirmed	Commit 949a3f4
M-10	🟢 Mitigation Confirmed	Commit b3b005e
M-12	🟢 Mitigation Confirmed	Commit 8d603d3
M-13	🟢 Mitigation Confirmed	Commit 14bb7cc
M-18	🟢 Mitigation Confirmed	Commit 30f90cc

🔗 Attacker can DoS liquidation by manipulating `currentTick` where the target position appears solvent

Submitted by [Valves](#).

Severity: Low

- `PanopticPool.sol` [L1526](#)
- `PanopticPool.sol` [L1461](#)

🔗 Finding description

The original issue reported that the `StaleOracle` check in `dispatchFrom()` reverted all operations (liquidations, force exercises, premium settlements) when `abs(currentTick - twapTick) > tickDeltaLiquidation`, blocking time-critical liquidations via deliberate spot price manipulation.

The team mitigated by moving the `StaleOracle` check inside the `solvent == numberOfTicks` branch, so it now only gates force exercises and premium settlements - not liquidations.

While this fixes the original `StaleOracle` DoS vector, the liquidation path remains vulnerable to the same manipulation vector but in a different check (solvency check itself). This attack was also explained in [this report](#).

The `dispatchFrom()` function checks solvency at 4 ticks:

```
int24[] memory atTicks = new int24[](4);
atTicks[0] = spotTick;    // stored EMA - not manipulable
atTicks[1] = twapTick;    // stored blended EMA - not manipulable
```

```
atTicks[2] = latestTick; // stored last tick - not manipulable
atTicks[3] = currentTick; // live Uniswap pool tick - MANIPULABLE
```

Liquidation requires `solvent == 0` (insolvent at all 4 ticks). If `solvent != 0 && solvent != numberOfTicks`, the code reverts with `NotMarginCalled`. An attacker can manipulate `currentTick` (the same way that he would have done with the previous issue with the `StaleOracle` check) to move the price to a tick where the target position appears solvent. This causes `solvent = 1` (insolvent at 3 oracle ticks, solvent at 1 manipulated tick), triggering:

```
} else {
    // otherwise, revert because the account is not fully margin called
    revert Errors.NotMarginCalled();
}
```

Recommended mitigation steps

Allow liquidation, even when `currentTick` is solvent, but all other ticks are not.

Disclosures

C4 audits incentivize the discovery of exploits, vulnerabilities, and bugs in smart contracts. Security researchers are rewarded at an increasing rate for finding higher-risk issues. Audit submissions are judged by a knowledgeable security researcher and disclosed to sponsoring developers. C4 does not conduct formal verification regarding the provided code but instead provides final verification.

C4 does not provide any guarantee or warranty regarding the security of this project. All smart contract software should be used at the sole risk and responsibility of users.

Top